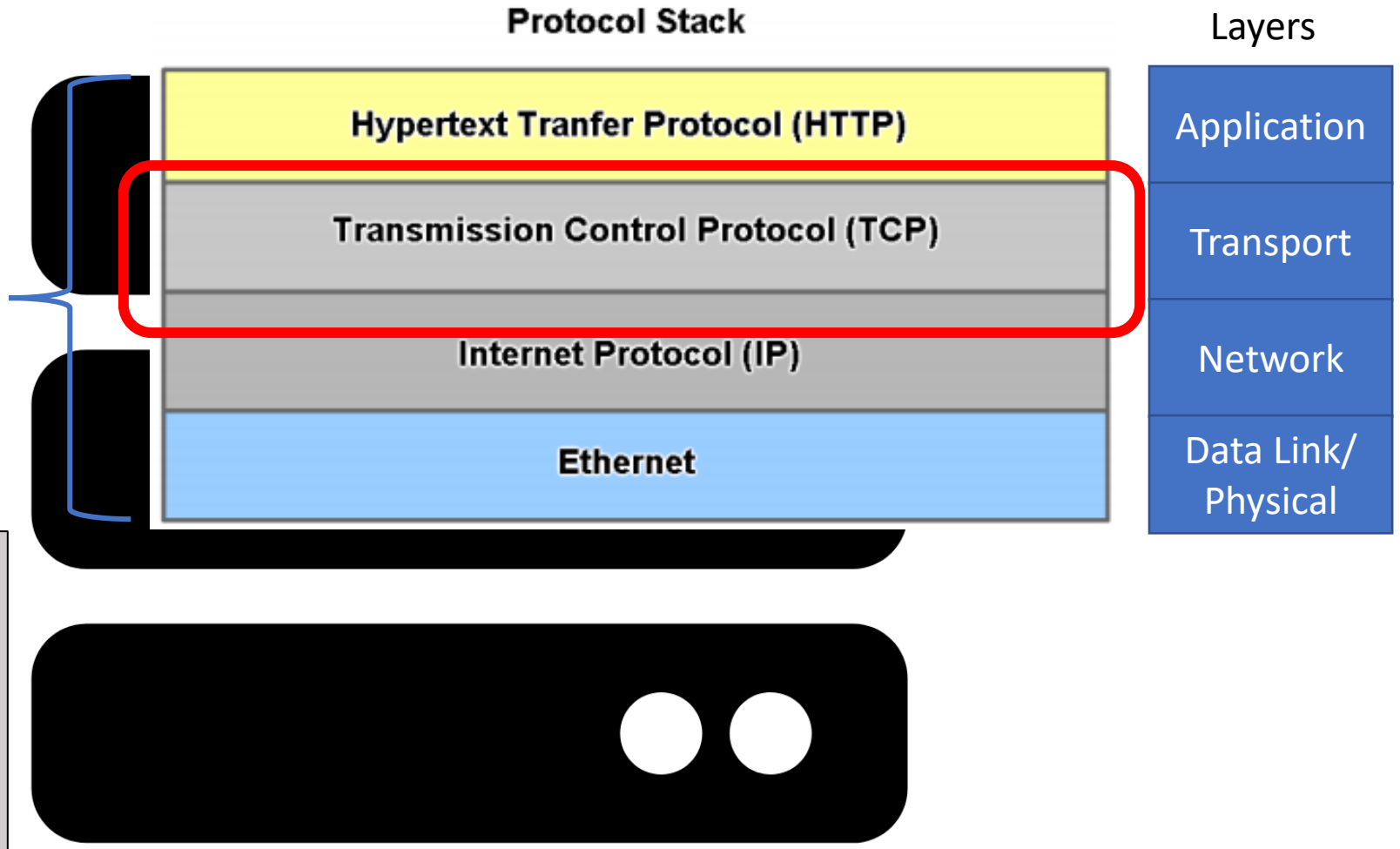
Transport Layer
TCP/UDP

By: Mujahid Tabassum - 2026

Recap: Layered Model Network Comms

Web Server

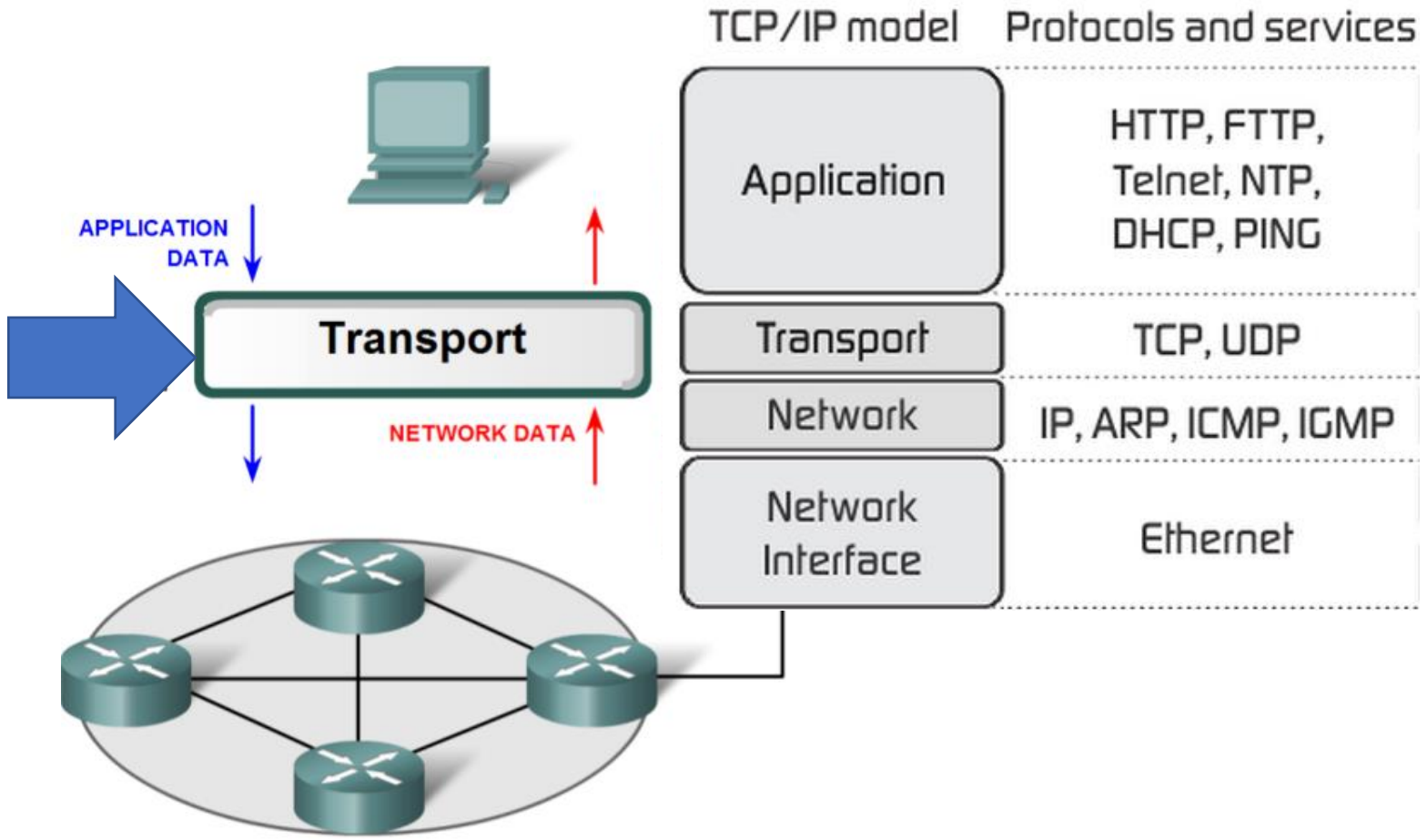


Benefits of Layered Model:

- assists in protocol design
- fosters competition
- changes in one layer do not affect other layers
- provides a common language

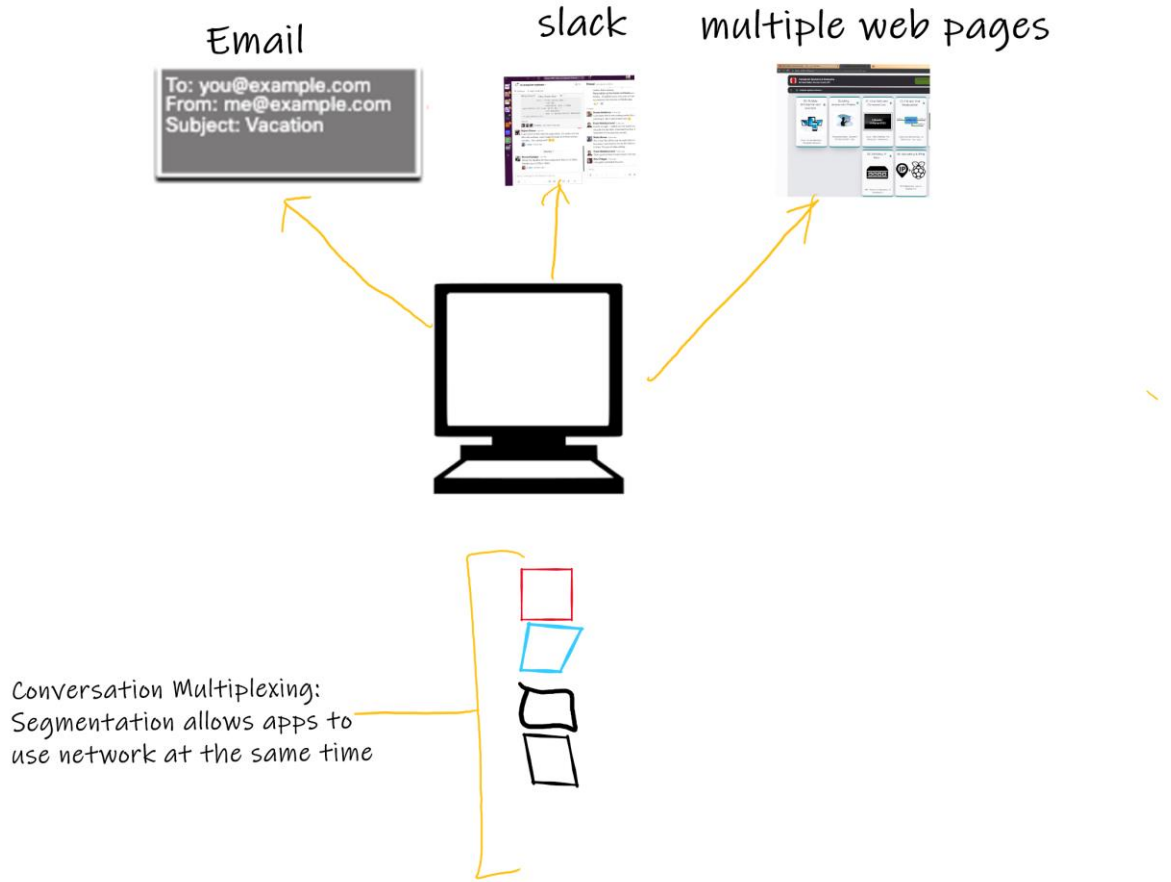
Transport Layer Role

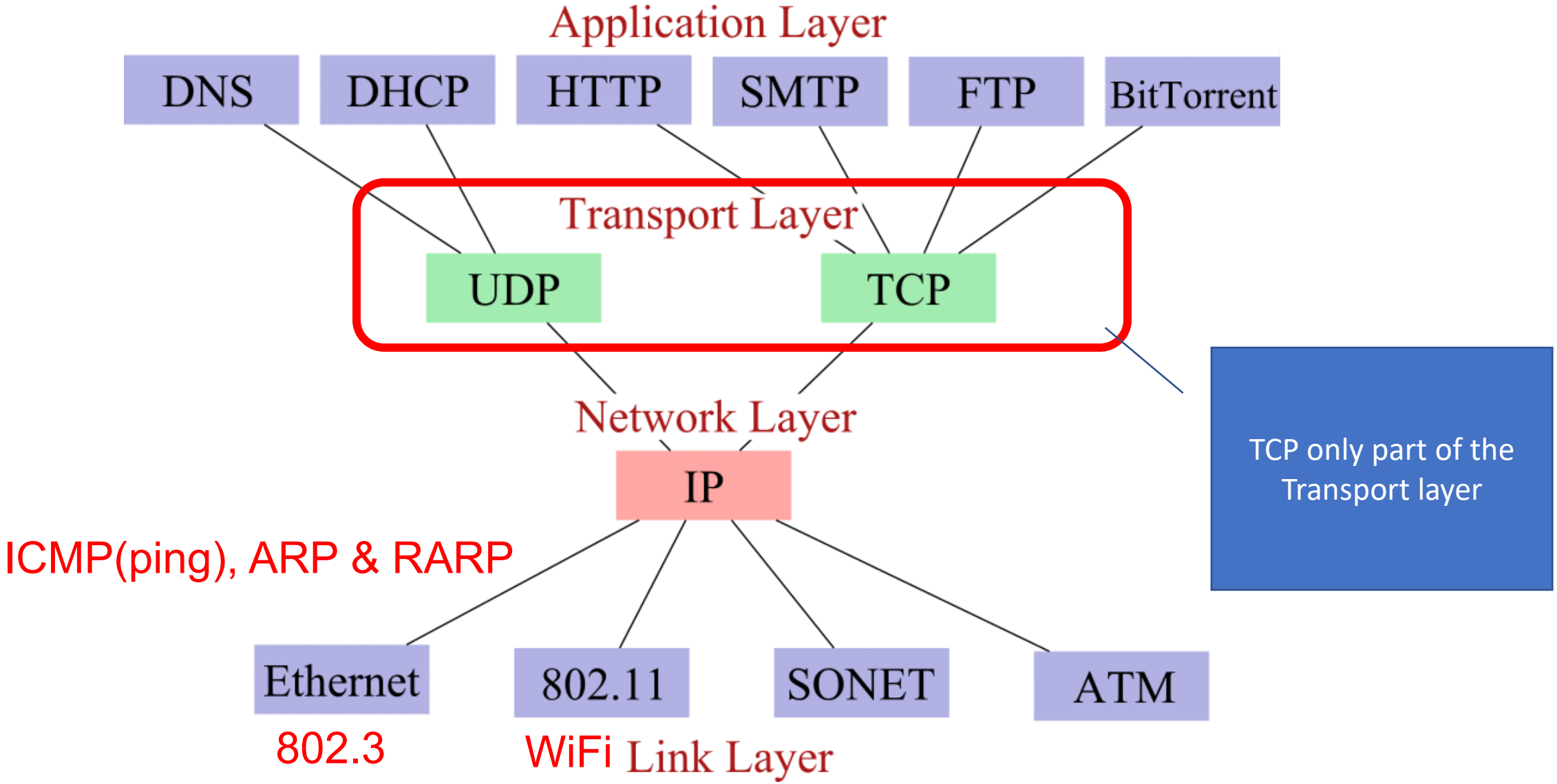
- Provides logical communications between applications running on different hosts.
- Provides the link between the application layer and the lower layers that are responsible for network transmission.
  - Prepares application data for transport over a network
  - Processes network data for use by apps



Transport Layer Purpose

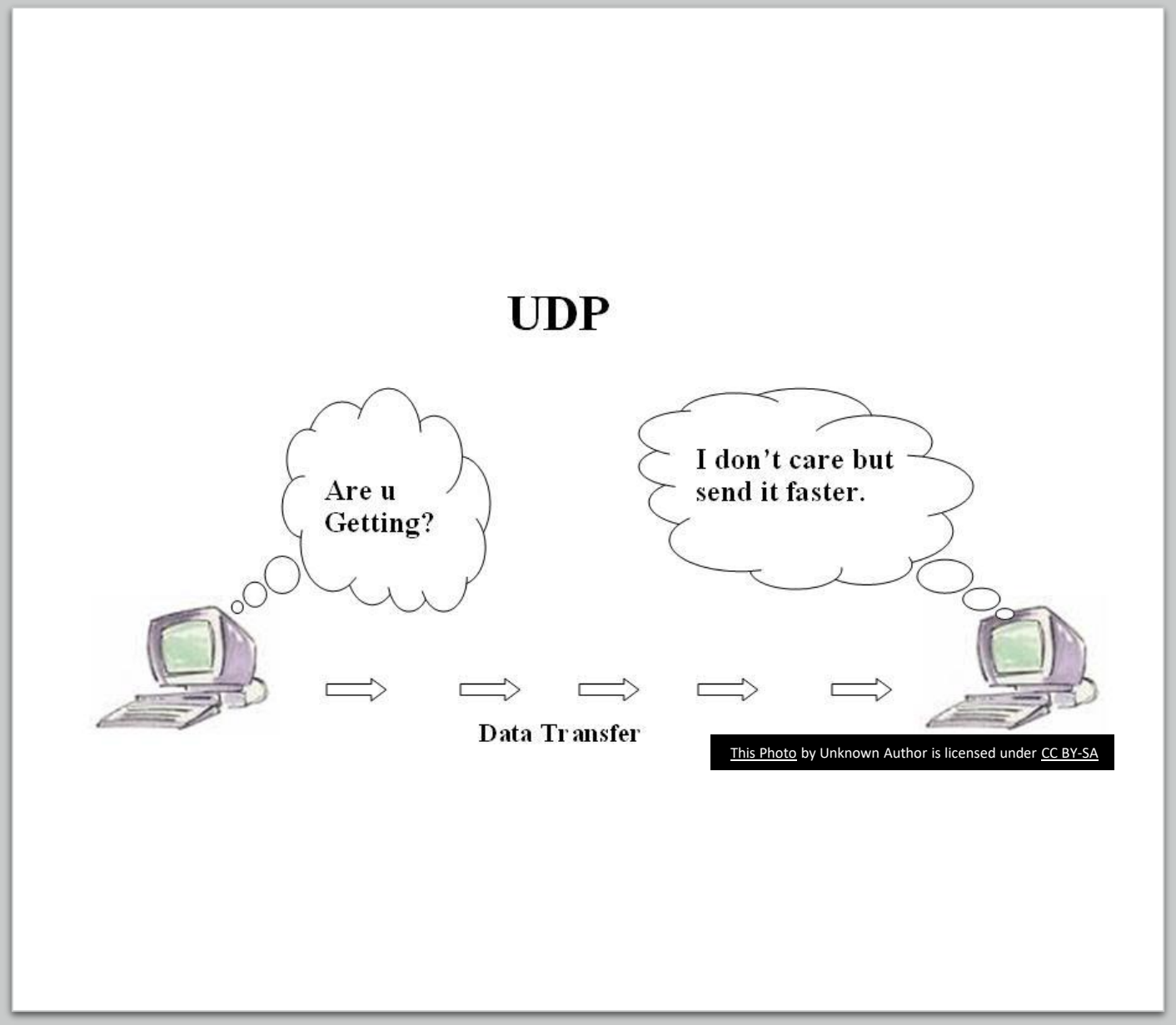
- Segments data and reassembles segments into various communication streams:
  - **Tracks individual communication between apps on source and destination hosts**
    - A host can have multiple simultaneous networked apps (e.g. browser, slack etc.
  - **Segments data and manages each piece**
    - The Transport layer segments data from the Application layer
    - Each piece of application data requires headers to indicate to which communication it is associated with
  - **Reassembles segments into application data**
    - At a receiving host, individual pieces of data must also be reconstructed into a complete data stream that is useful to the Application layer.
  - **Identifies different applications**
    - Transport layer must be able to identify target apps: Uses Port Numbers




Application Layer
DNS
DHCP
HTTP
SMTP
FTP
BitTorrent
Transport Layer
UDP
TCP
Network Layer
IP
ICMP(ping), ARP & RARP
TCP only part of the Transport layer
Ethernet
802.11
SONET
ATM
802.3
WiFi
Link Layer

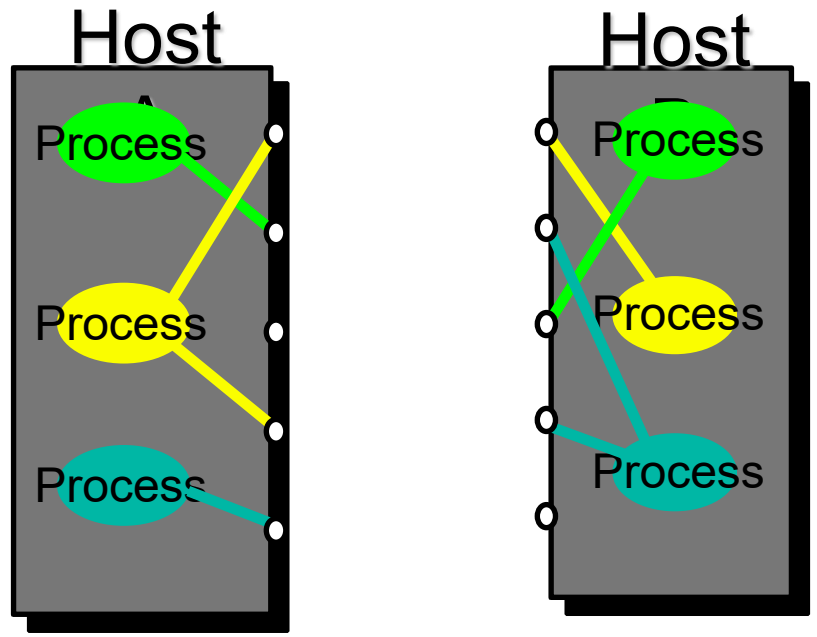
UDP - User Datagram Protocol

- UDP is a transport protocol, that provides the basic functions for delivering datagrams between the appropriate applications,
  - communication between **processes**
- UDP is a connectionless best-effort delivery
  - there is no acknowledgment that the data is received at the destination.
- UDP uses ***ports*** to provide communication services to individual apps/processes.



Ports

- TCP/IP uses an abstract destination point called a protocol port.
- Ports are identified by a positive integer.
  - **Port number & IP address allow any process/application in any computer on Internet to be uniquely identified**
- Operating systems provide a mechanism that apps & processes use to specify a port.



Ports

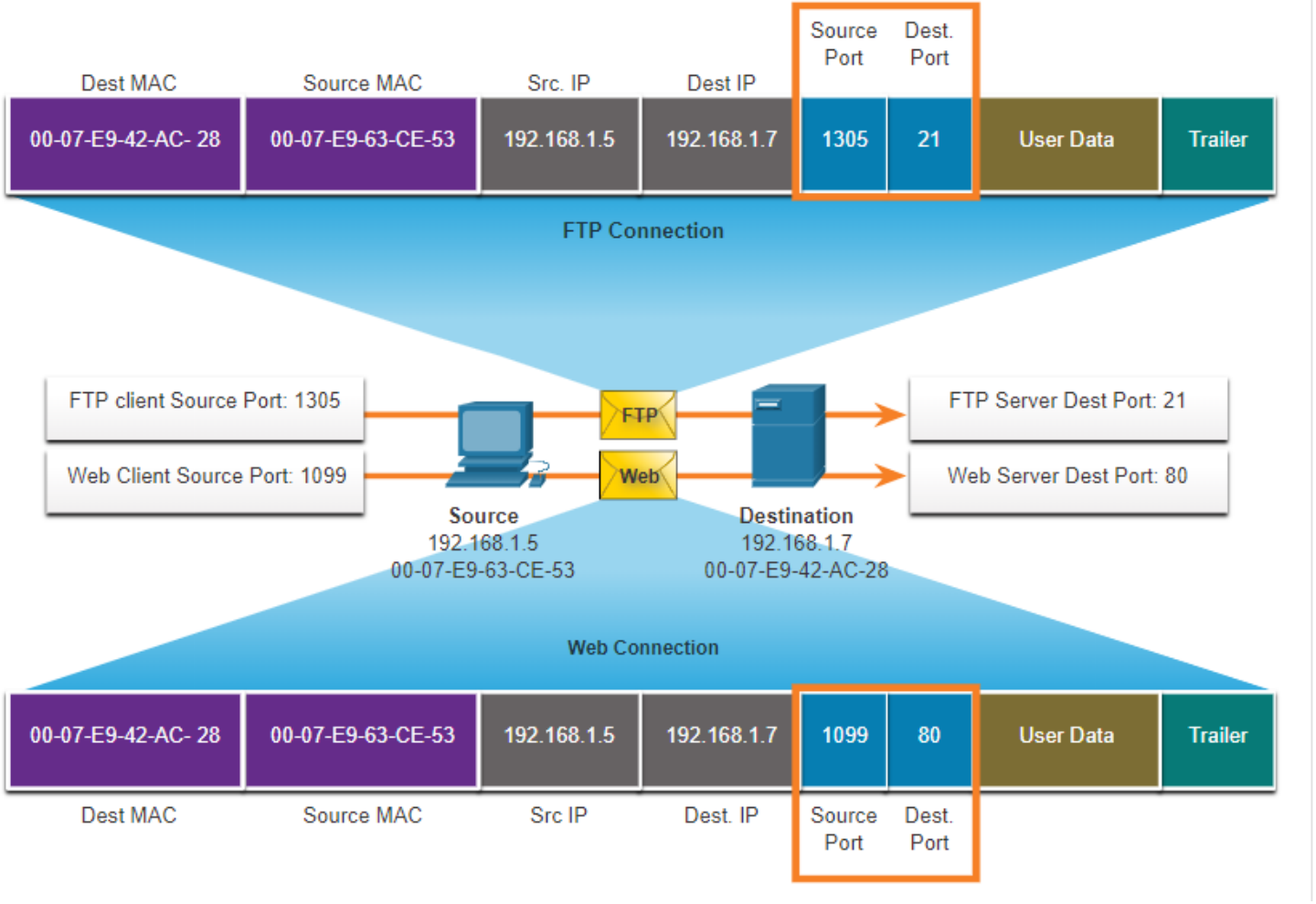
- Source/destination port: port numbers identify sending & receiving processes
- Ports can be static or dynamic
  - Static (< 1024) assigned centrally, known as well known ports
  - Dynamic (can be used by any computer application program to communicate with any other application program, 49152-65535 for Windows)

Ports: Multiple communications

- Port numbers used in TCP and UDP to manage multiple, simultaneous conversations.
- The **source port number** is associated with the **originating application** on the local host whereas the **destination port number** is associated with the **destination application** on the remote host.

Ports: Socket Pairs

- The source and destination ports are placed within the segment
- The segments are then encapsulated within an IP packet.
- The combination of the source IP address and source port number, or the destination IP address and destination port number is known as a **socket**.
- Sockets enable multiple processes, running on a client, to distinguish themselves from each other, and multiple connections to a server process to be distinguished from each other



Ports: Groups

Port Group	Number Range	Description
Well-known Ports	**0 to 1,023**	•These port numbers are reserved for common or popular services and applications such as web browsers, email clients, and remote access clients. •Defined well-known ports for common server applications enables clients to easily identify the associated service required.
Registered Ports	**1,024 to 49,151**	•These port numbers are assigned by IANA to a requesting entity to use with specific processes or applications. •These processes are primarily individual applications that a user has chosen to install, rather than common applications that would receive a well-known port number. •For example, Cisco has registered port 1812 for its RADIUS server authentication process.
Private and/or **Dynamic Ports**	**49,152 to 65,535**	•These ports are also known as *ephemeral ports*. •The client's OS usually assign port numbers dynamically when a connection to a service is initiated. •The dynamic port is then used to identify the client application during communication.

Well-Known Port Numbers

Port Number	Protocol	Application
20	TCP	File Transfer Protocol (FTP) - Data
21	TCP	File Transfer Protocol (FTP) - Control
22	TCP	Secure Shell (SSH)
23	TCP	Telnet
25	TCP	Simple Mail Transfer Protocol (SMTP)
53	UDP, TCP	Domain Name Service (DNS)
67	UDP	Dynamic Host Configuration Protocol (DHCP) - Server
68	UDP	Dynamic Host Configuration Protocol - Client
69	UDP	Trivial File Transfer Protocol (TFTP)
80	TCP	Hypertext Transfer Protocol (HTTP)
110	TCP	Post Office Protocol version 3 (POP3)
143	TCP	Internet Message Access Protocol (IMAP)
161	UDP	Simple Network Management Protocol (SNMP)
443	TCP	Hypertext Transfer Protocol Secure (HTTPS)

The netstat command

- Unexplained TCP connections can pose a major security threat. Netstat is an important tool to verify connections.

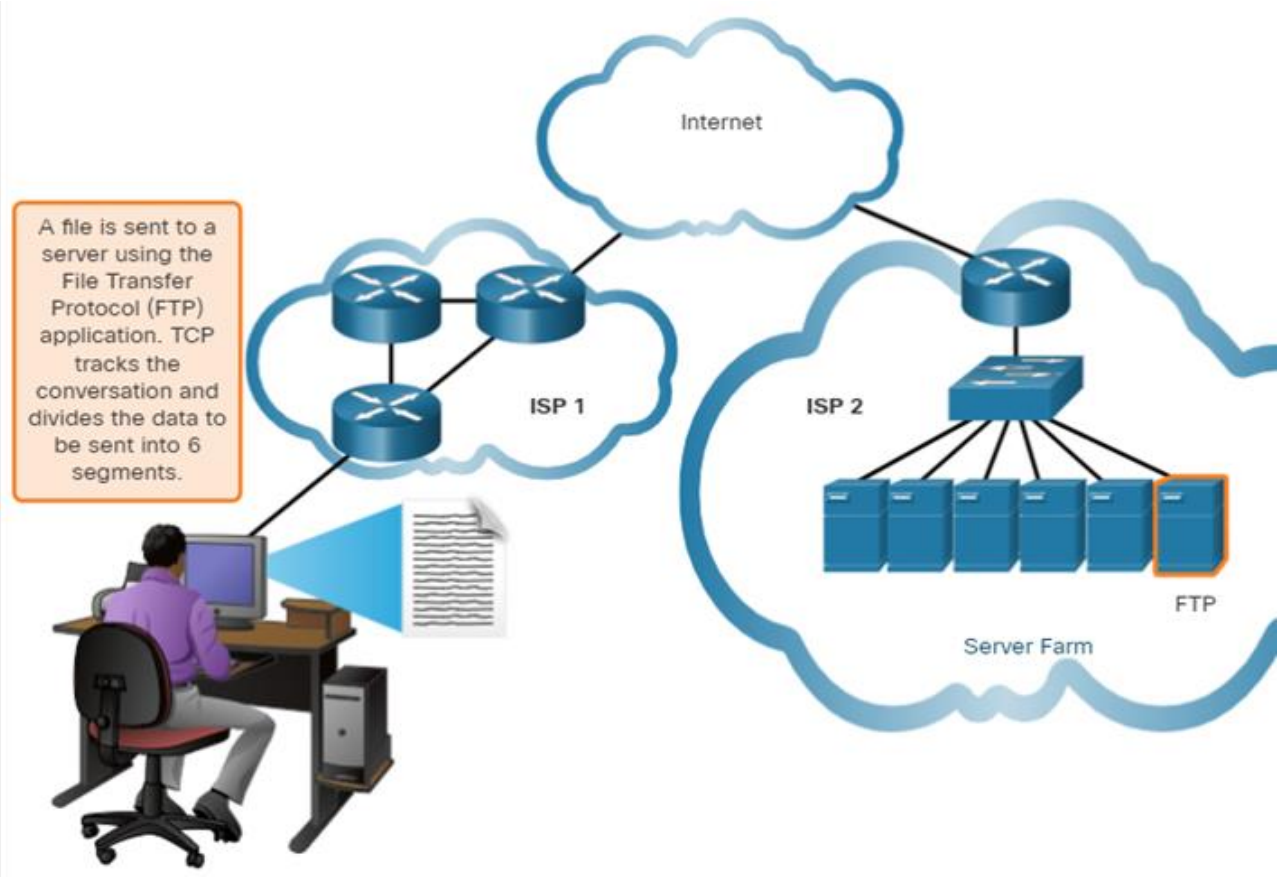
```
C:\> netstat
Active Connections
Proto Local Address                    Foreign Address                        State
TCP   192.168.1.124:3126          192.168.0.2:netbios-ssn          ESTABLISHED
TCP   192.168.1.124:3158          207.138.126.152:http                   ESTABLISHED
TCP   192.168.1.124:3159          207.138.126.169:http                   ESTABLISHED
TCP   192.168.1.124:3160          207.138.126.169:http                   ESTABLISHED
TCP   192.168.1.124:3161          sc.msn.com:http                        ESTABLISHED
TCP   192.168.1.124:3166          www.cisco.com:http                     ESTABLISHED
```

TCP Overview and Communication Process

Transmission Control Protocol (TCP)

TCP provides reliability and flow control. TCP basic operations:

- Number and track data segments transmitted to a specific host from a specific application
- Acknowledge received data
- Retransmit any unacknowledged data after a certain amount of time
- Sequence data that might arrive in wrong order
- Send data at an efficient rate that is acceptable by the receiver



Choosing Transport Layer Protocols…

- Application developers choose based on the nature of the app

- IP Telephony
- Video Streaming
- Frequent Sensor data
- Online Gaming(Fortnite/Call of Duty)

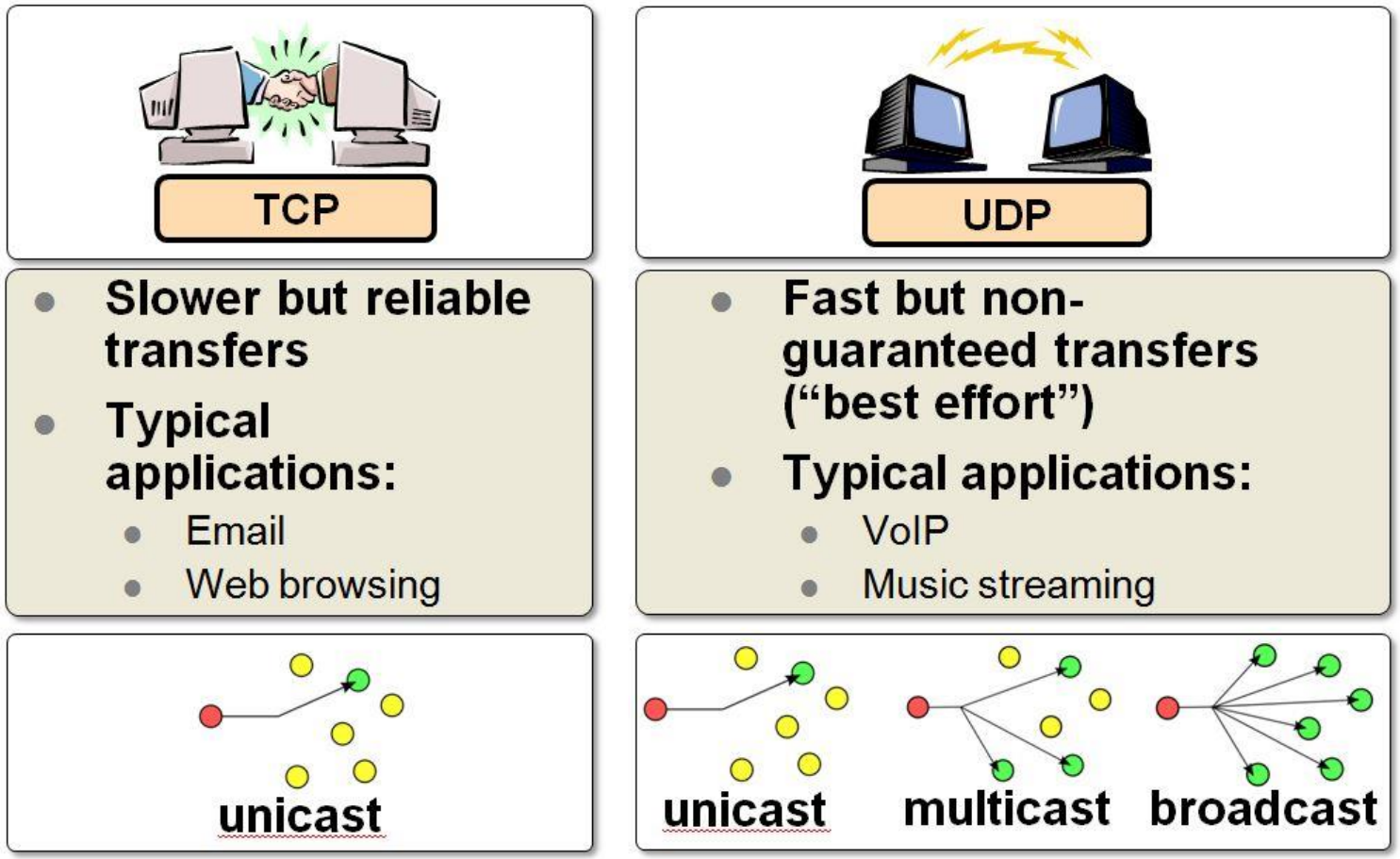
Requirements:
- Fast
- Low overhead
- No need for acknowedgements
- No need to resend lost data
- Delivers data as it arrives.

- Email(SMTP/POP)
- Web Pages(HTTP)
- Command data(turn on Heating)

Requirements:
- Reliable
- Acknowedge data
- Resend lost data
- Delivers data in order sent.

Choosing Transport Layer Protocols…

Q: Which protocol is better ?

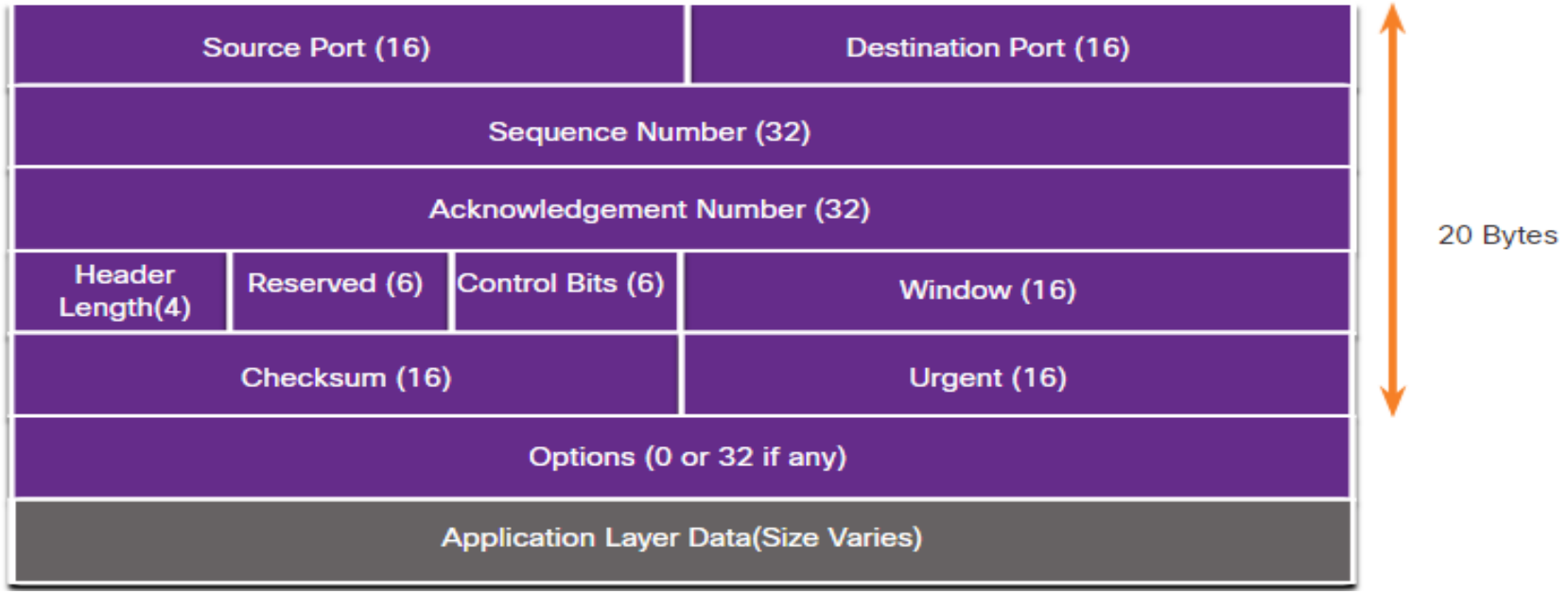


TCP Features and Datagram Format

- Establishes a Session
- Ensures Reliable Delivery
- Provides Same-Order Delivery
- Supports Flow Control

TCP is a stateful protocol which means it keeps track of the state of the communication session.

TCP records which information it has sent, and which information has been acknowledged.

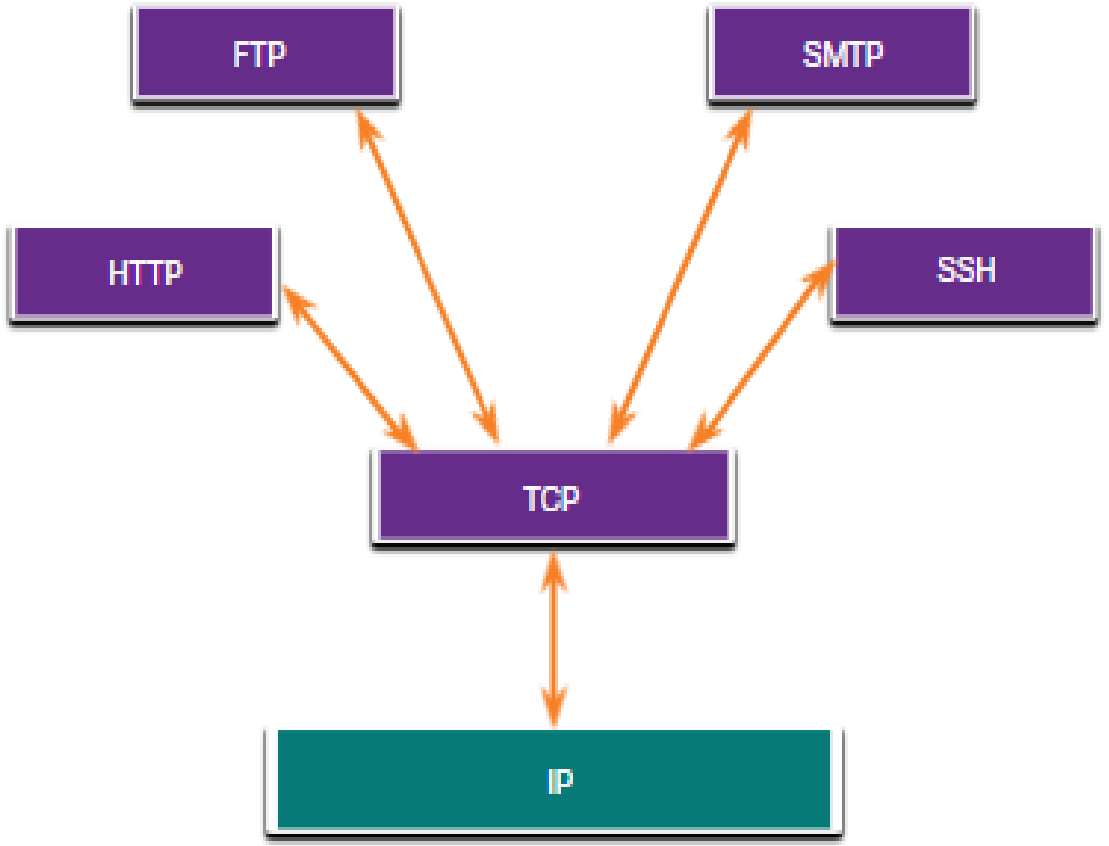


TCP Header Fields

TCP Header Field	Description
Source Port	A 16-bit field used to identify the source application by port number.
Destination Port	A 16-bit field used to identify the destination application by port number.
Sequence Number	A 32-bit field used for data reassembly purposes.
Acknowledgment Number	A 32-bit field used to indicate that data has been received and the next byte expected from the source.
Header Length	A 4-bit field known as "data offset" that indicates the length of the TCP segment header.
Reserved	A 6-bit field that is reserved for future use.
Control bits	A 6-bit field used that includes bit codes, or flags, which indicate the purpose and function of the TCP segment.
Window size	A 16-bit field used to indicate the number of bytes that can be accepted at one time.
Checksum	A 16-bit field used for error checking of the segment header and data.
Urgent	A 16-bit field used to indicate if the contained data is urgent.

Applications that use TCP

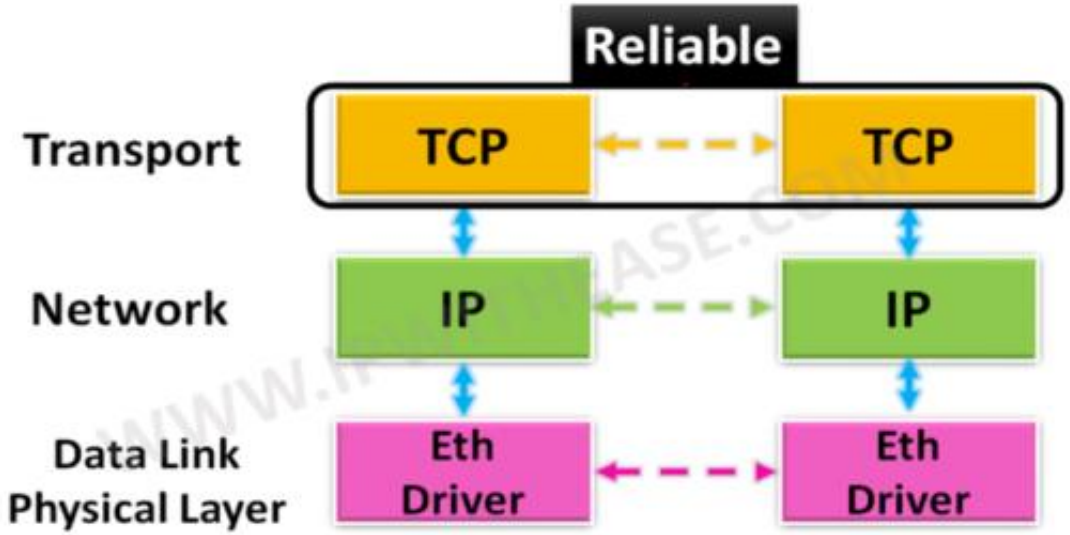
TCP handles all tasks associated with dividing the data stream into segments, providing reliability, controlling data flow, and reordering segments.



TCP - *Transmission Control Protocol*

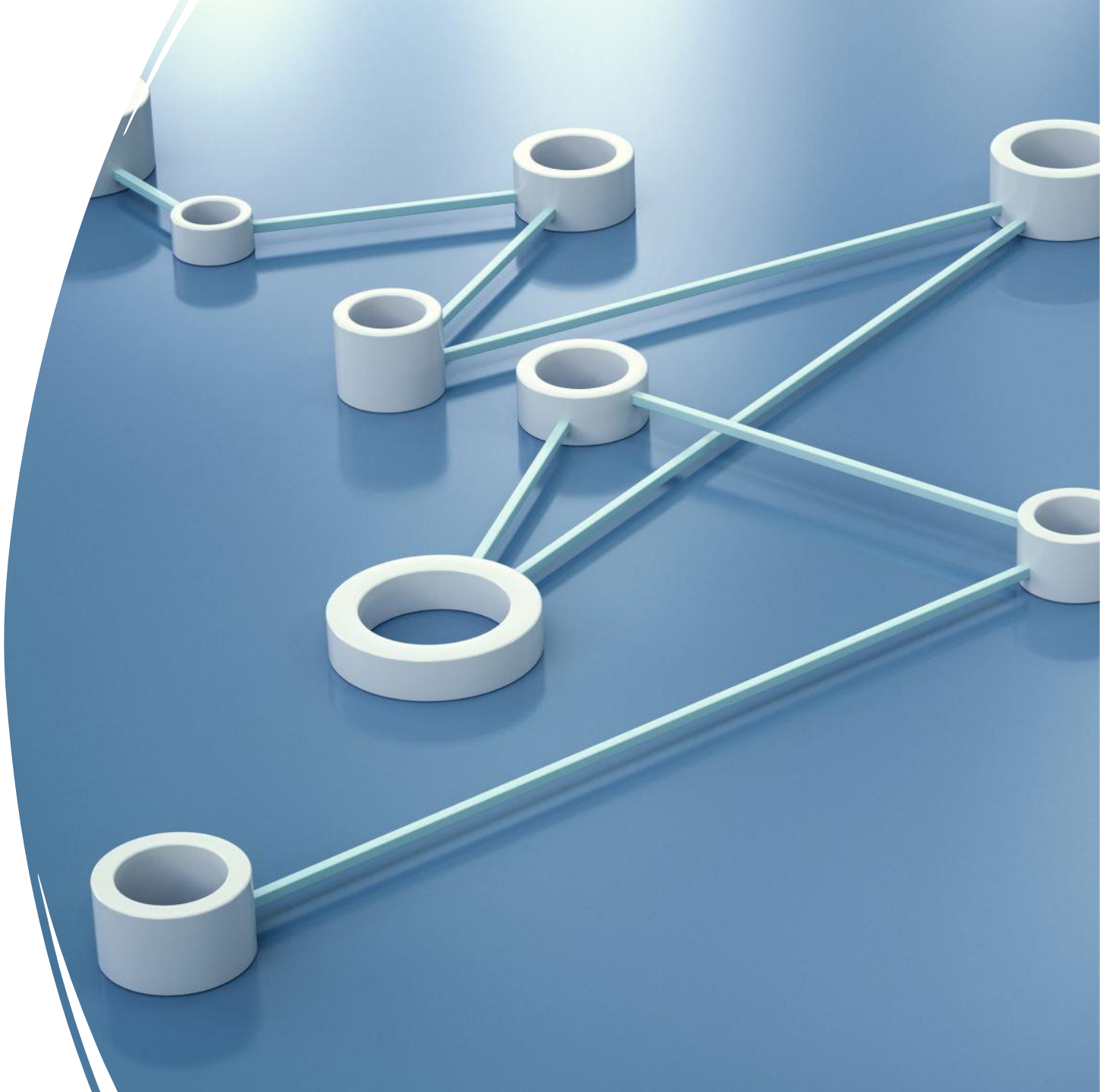
- TCP is a transport layer protocol supported by TCP/IP stack.

- TCP provides:
  - Connection-oriented
  - Reliable
  - Full-duplex
  - Byte-Stream



Connection-Oriented

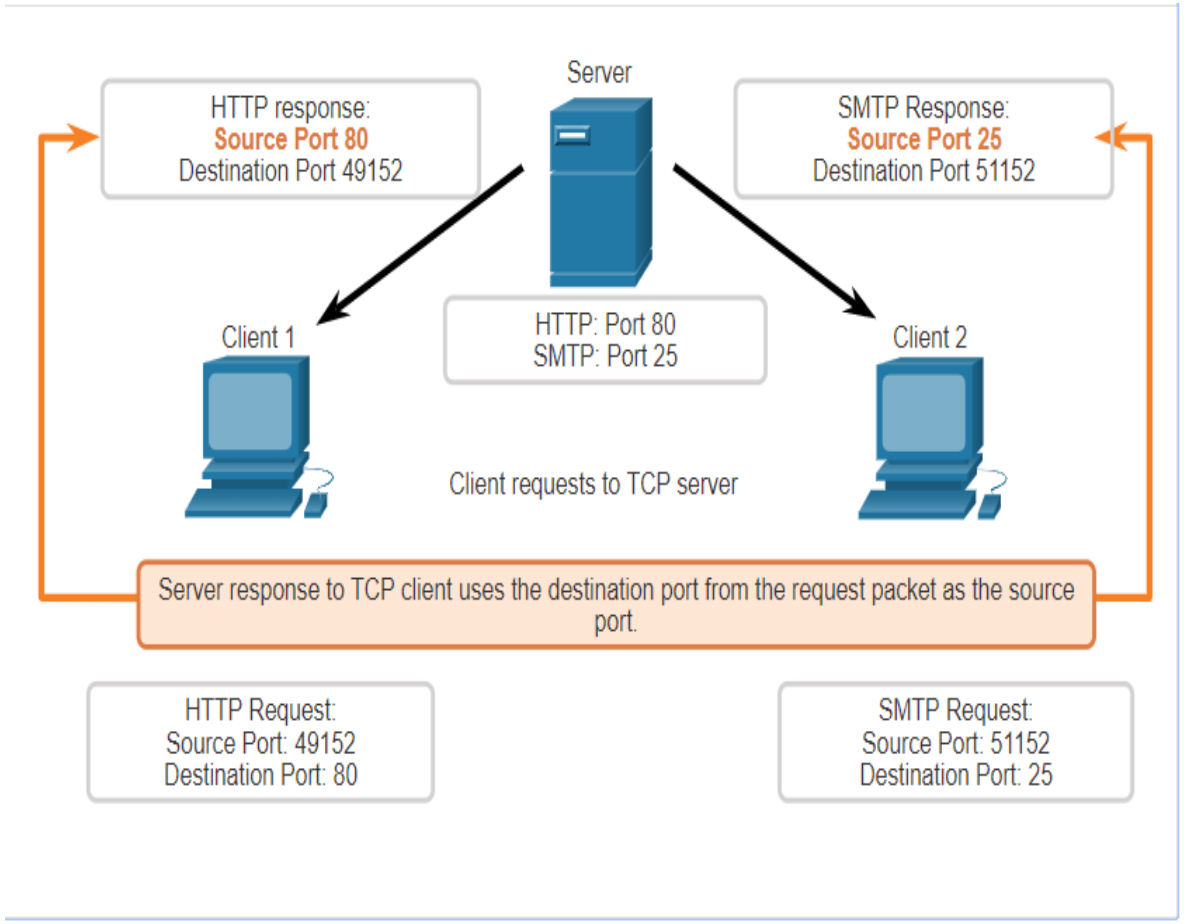
- *Connection oriented* means that a virtual connection is established before any user data is transferred.
- If the connection cannot be established, the user program is notified (finds out).
- If the connection is ever interrupted, the user program(s) is finds out there is a problem.



TCP Server Processes

Each application process running on a server is configured to use a port number.

- An individual server cannot have two services assigned to the same port number within the same transport layer services.
- An active server application assigned to a specific port is considered open, which means that the transport layer accepts, and processes segments addressed to that port.
- Any incoming client request addressed to the correct socket is accepted, and the data is passed to the server application.

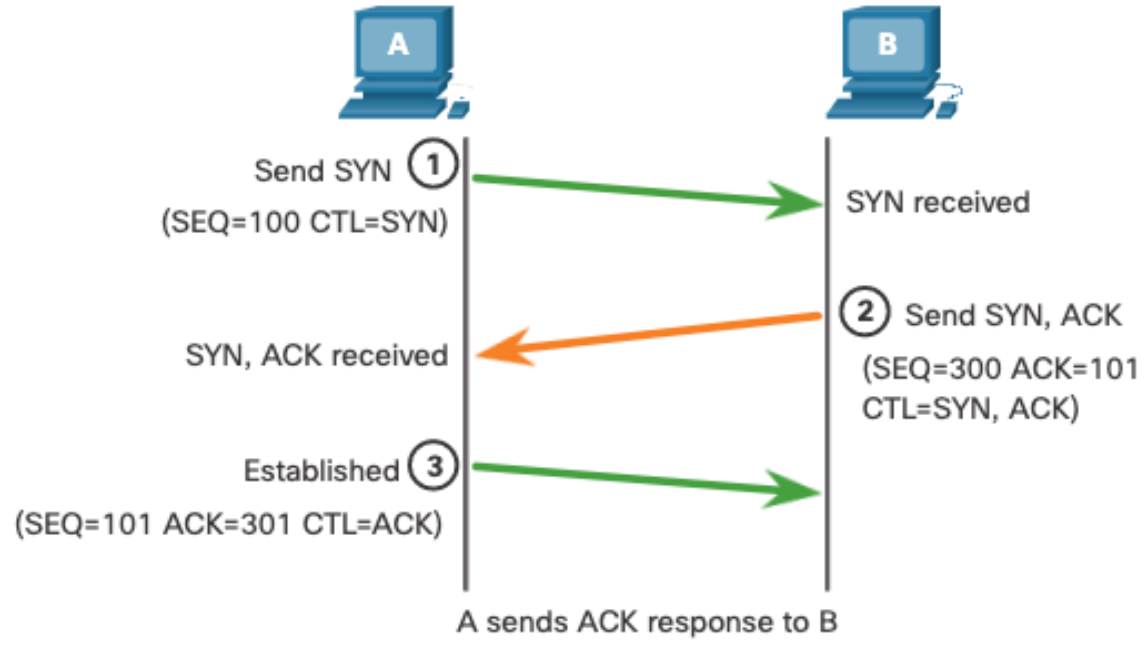


TCP Connection Establishment

Step 1: The initiating client requests a client-to-server communication session with the server.

Step 2: The server acknowledges the client-to-server communication session and requests a server-to-client communication session.

Step 3: The initiating client acknowledges the server-to-client communication session.



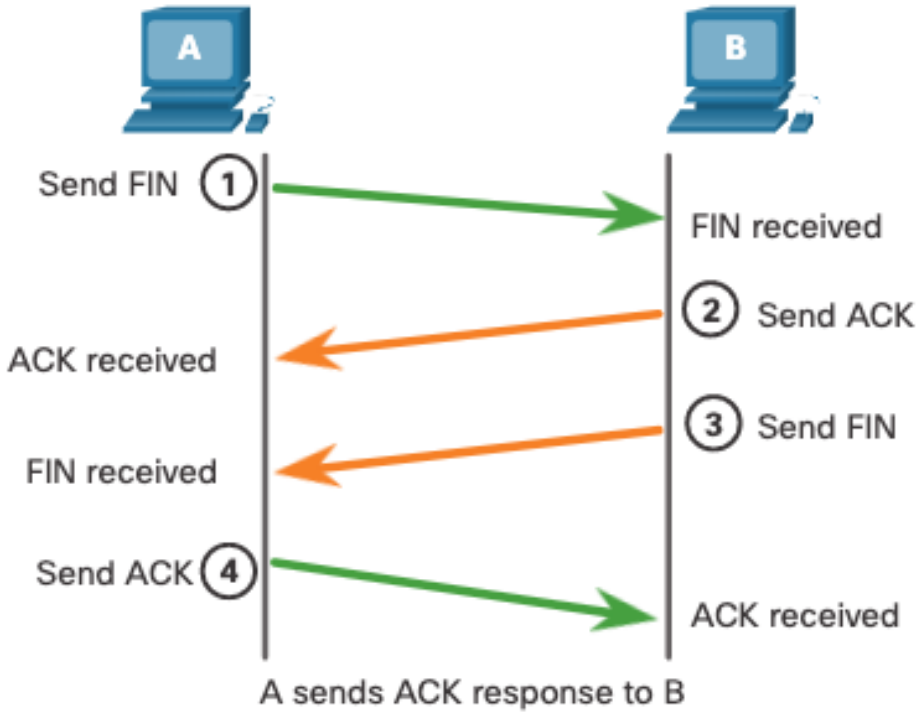
TCP Connection Termination

Step 1: When the client has no more data to send in the stream, it sends a segment with the FIN flag set.

Step 2: The server sends an ACK to acknowledge the receipt of the FIN to terminate the session from client to server.

Step 3: The server sends a FIN to the client to terminate the server-to-client session.

Step 4: The client responds with an ACK to acknowledge the FIN from the server.



TCP Three Way Handshake Analysis

Functions of the Three-Way Handshake:

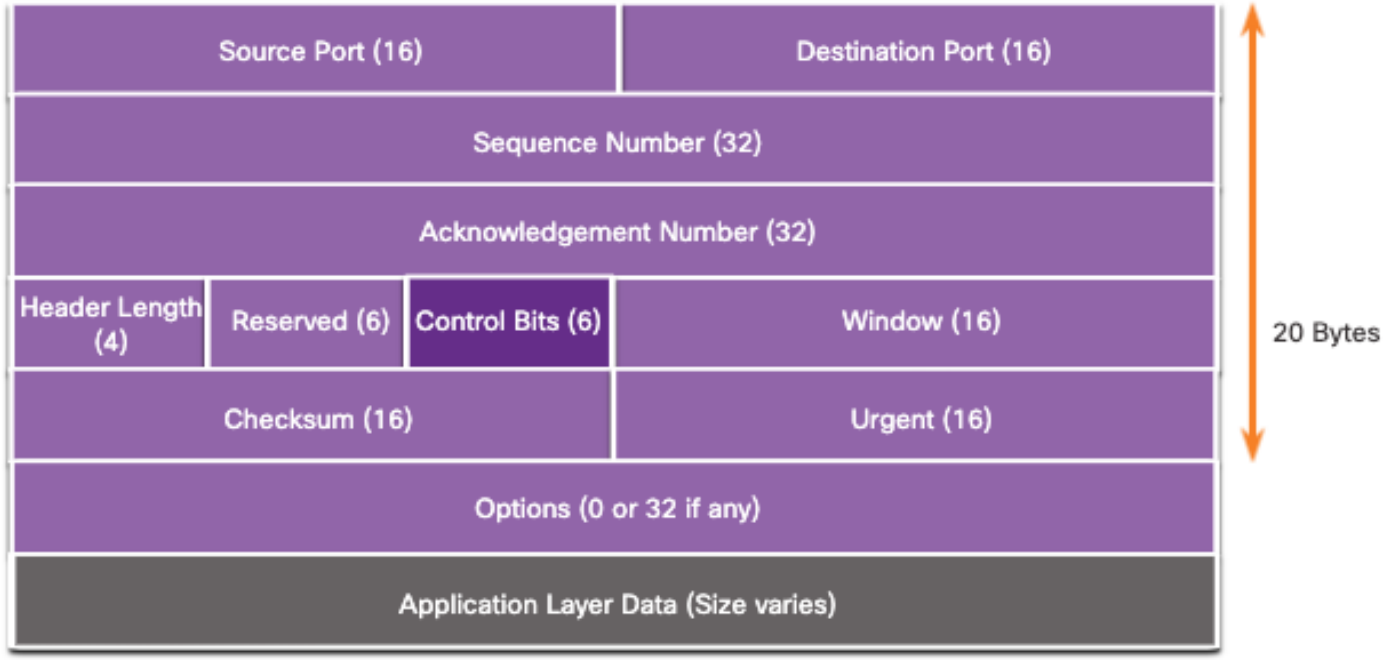
- It establishes that the destination device is present on the network.
- It verifies that the destination device has an active service and is accepting requests on the destination port number that the initiating client intends to use.
- It informs the destination device that the source client intends to establish a communication session on that port number.

After the communication is completed, the sessions are closed, and the connection is terminated. The connection and session mechanisms enable TCP reliability function.

TCP Three Way Handshake Analysis

The six control bit flags are as follows:

- URG - Urgent pointer field significant
- ACK - Acknowledgment flag used in connection establishment and session termination
- PSH - Push function
- RST - Reset the connection when an error or timeout occurs
- SYN - Synchronize sequence numbers used in connection establishment
- FIN - No more data from sender and used in session termination



TCP Connection Creation

- A *server* accepts a connection.
  - Must be looking for new connections!

- A *client* requests a connection.
  - Must *know* where the server is!

Client Starts

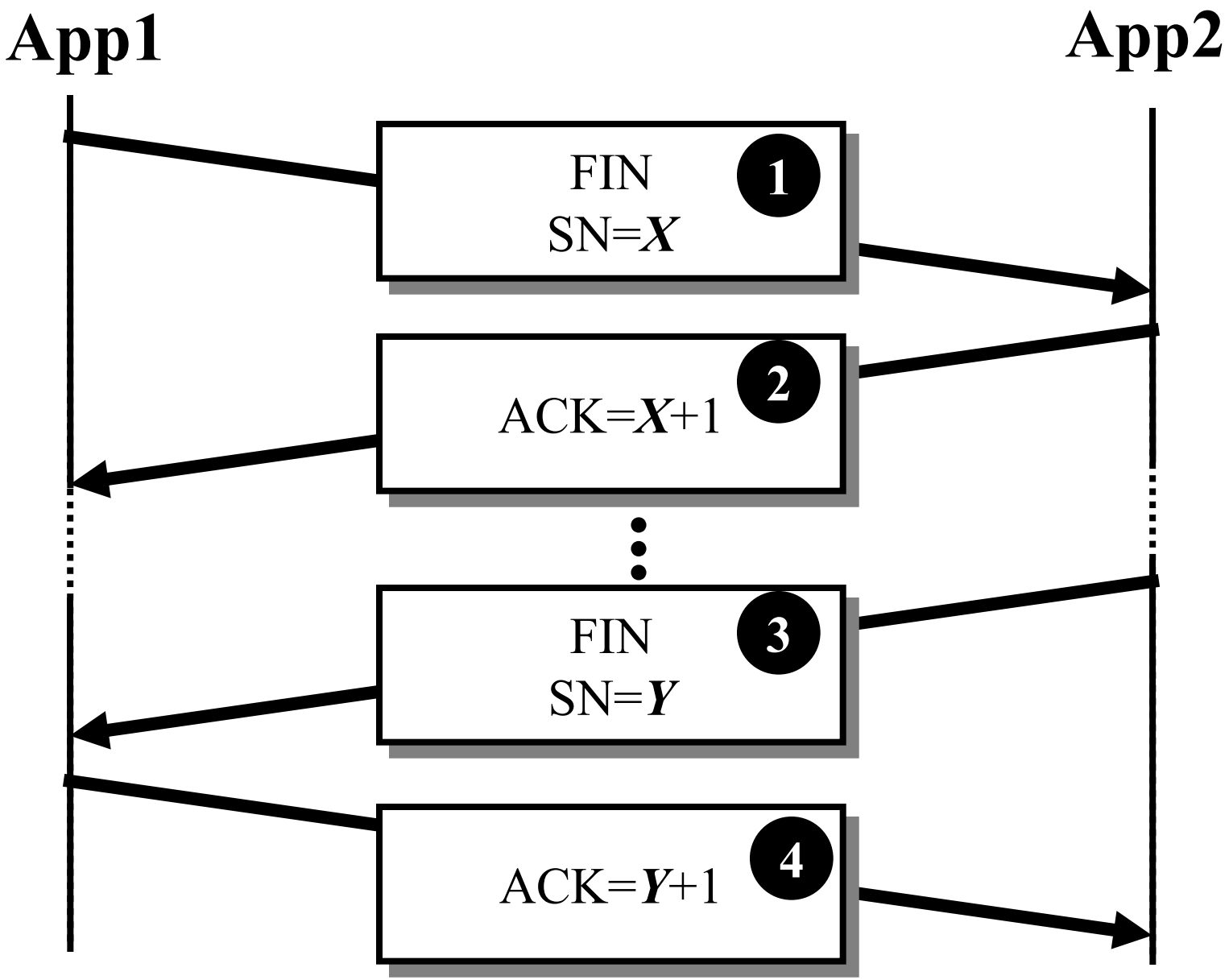
- A client starts by sending a SYN segment with the following information:
  - Client's ISN (generated pseudo-randomly)
  - Maximum Receive Window for client.
  - Optionally (but usually) MSS (largest segment accepted).
  - No payload! (Only TCP headers)

Sever Response

- When a waiting server sees a new connection request, the server sends back a SYN segment with:
  - Server's ISN (generated pseudo-randomly)
  - Acknowledgement Number is Client ISN+1
  - Maximum Receive Window for server.
  - Optionally (but usually) MSS
  - No payload! (Only TCP headers)

Finally

- When the Server's SYN is received, the client sends back an ACK with:
  - Acknowledgement Number is Server's ISN+1


App1
App2
FIN
SN=X
1
ACK=X+1
2
FIN
SN=Y
3
ACK=Y+1
4

TCP Termination

1. App1: “I have no more data for you”.

2. App2: “OK, I understand you are done sending.”

   dramatic pause…

3. App2: “OK - Now I’m also done sending data”.

4. App1: “Roger, Over and Out, Goodbye, Astalavista Baby, Adios, It’s been real ...”

   camera fades to black ...

TCP Data and ACK

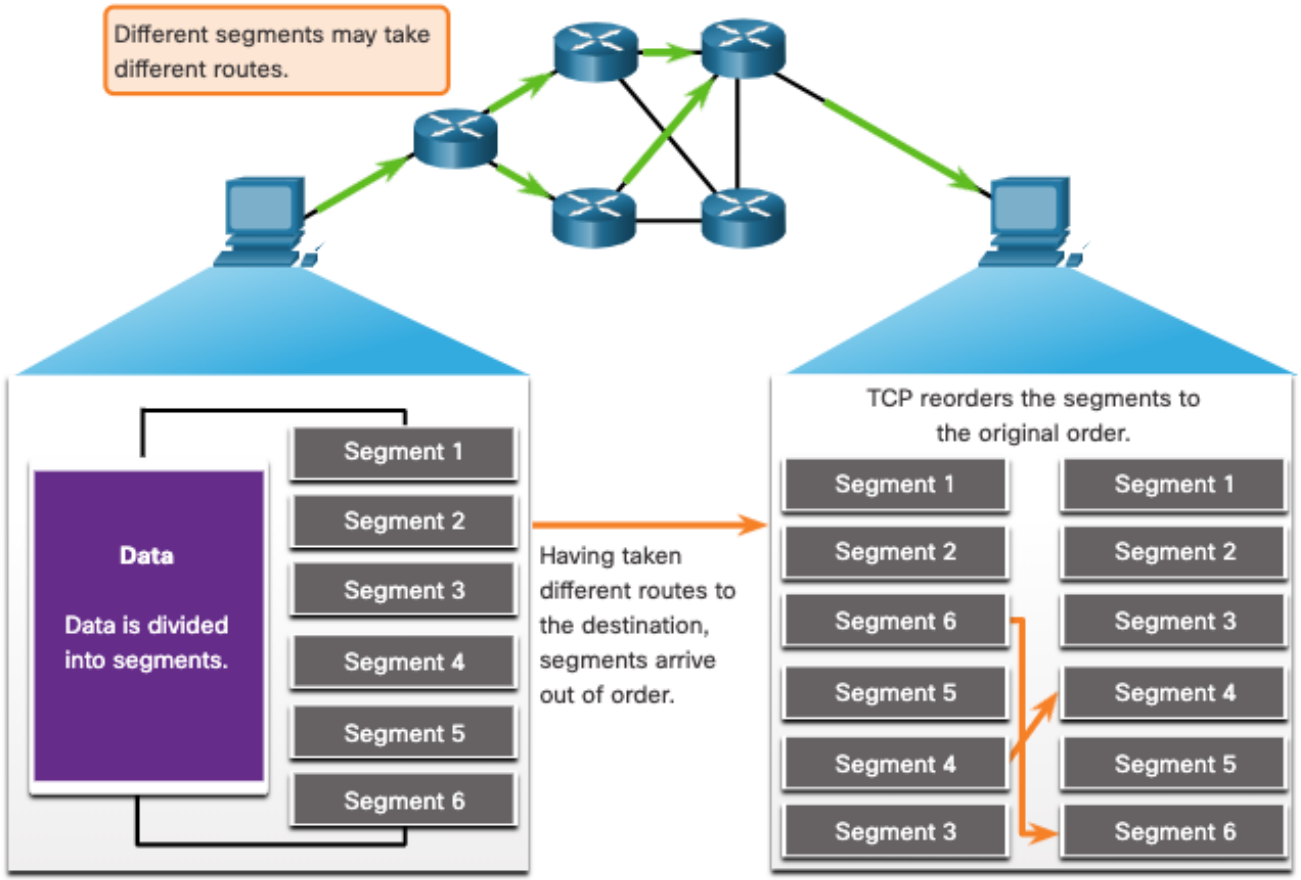
- Once the connection is established, data can be sent.
- Each data segment includes a sequence number identifying the first byte in the segment.
- Each segment (data or empty) includes a request number indicating what data has been received.

Reliable

- *Reliable* means that every transmission of data is acknowledged by the receiver.
- Reliable does not mean that things don't go wrong, it means that we find out when things go wrong.
- If the sender does not receive acknowledgement within a specified amount of time, the sender retransmits the data.

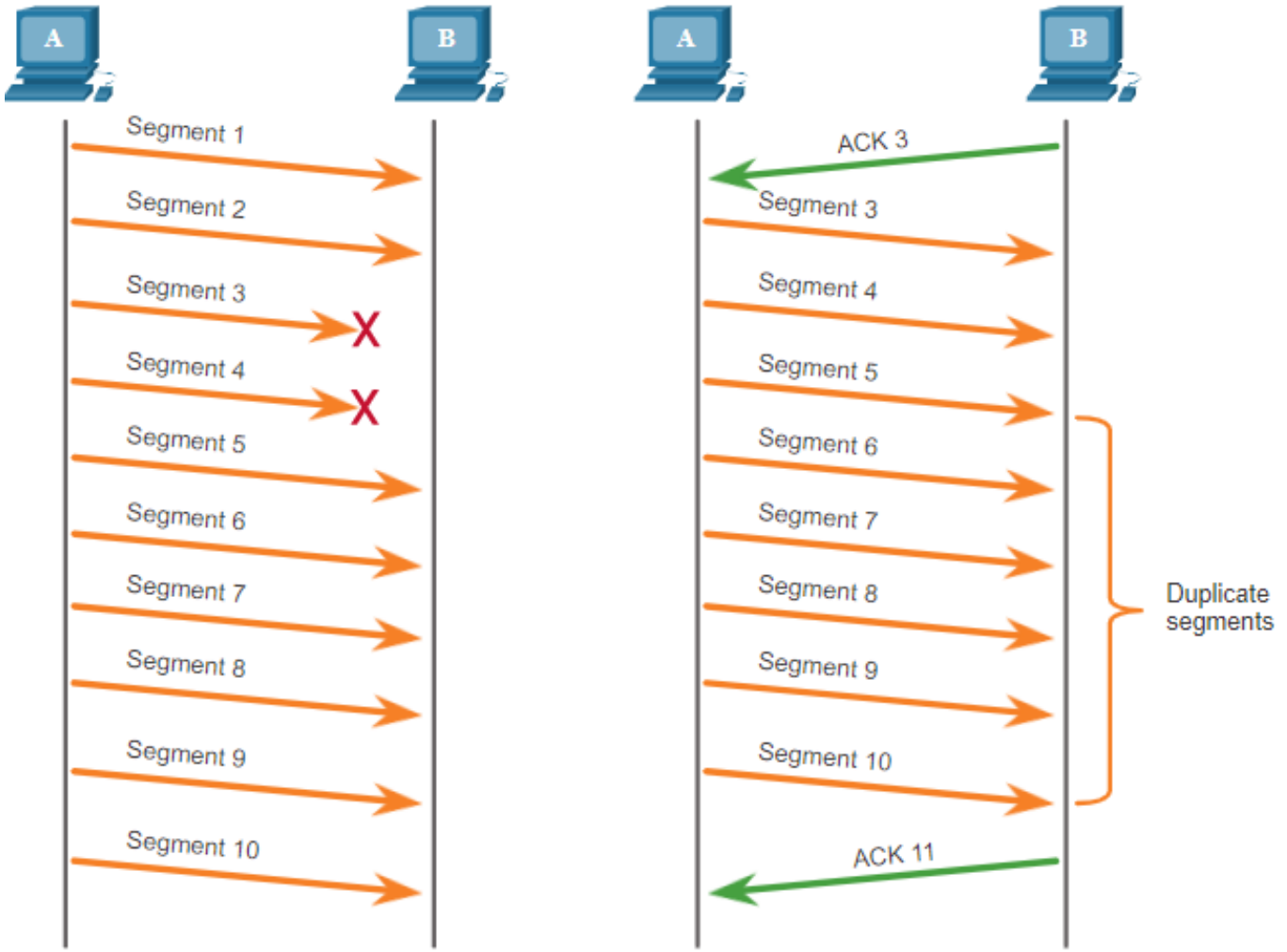
TCP Reliability - Guaranteed and Ordered Delivery

- TCP can also help maintain the flow of packets so that devices do not become overloaded.
- There may be times when TCP segments do not arrive at their destination or arrive out of order.
- All the data must be received and the data in these segments must be reassembled into the original order.
- Sequence numbers are assigned in the header of each packet to achieve this goal.



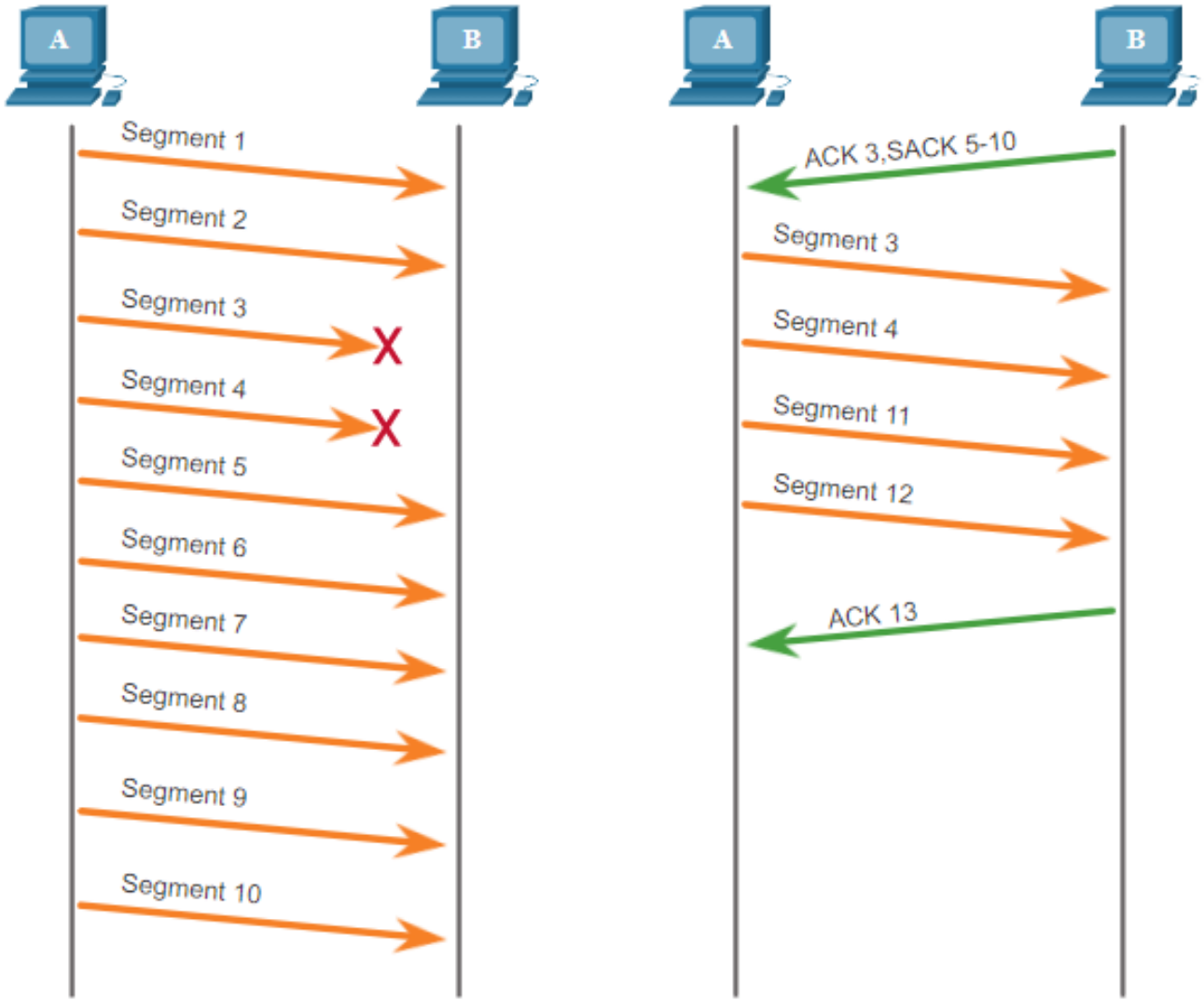
TCP Reliability – Data Loss and Retransmission

- No matter how well designed a network is, data loss occasionally occurs.
- TCP provides methods of managing these segment losses. Among these is a mechanism to retransmit segments for unacknowledged data.



TCP Reliability – Data Loss and Retransmission

- Host operating systems today typically employ an optional TCP feature called selective acknowledgment (SACK), negotiated during the three-way handshake.
- If both hosts support SACK, the receiver can explicitly acknowledge which segments (bytes) were received including any discontinuous segments.



Full Duplex

- TCP provides transfer in both directions (over a single virtual connection).
- To the application program these appear as 2 unrelated data streams, although TCP can piggyback control and data communication by providing control information (such as an ACK) along with user data.

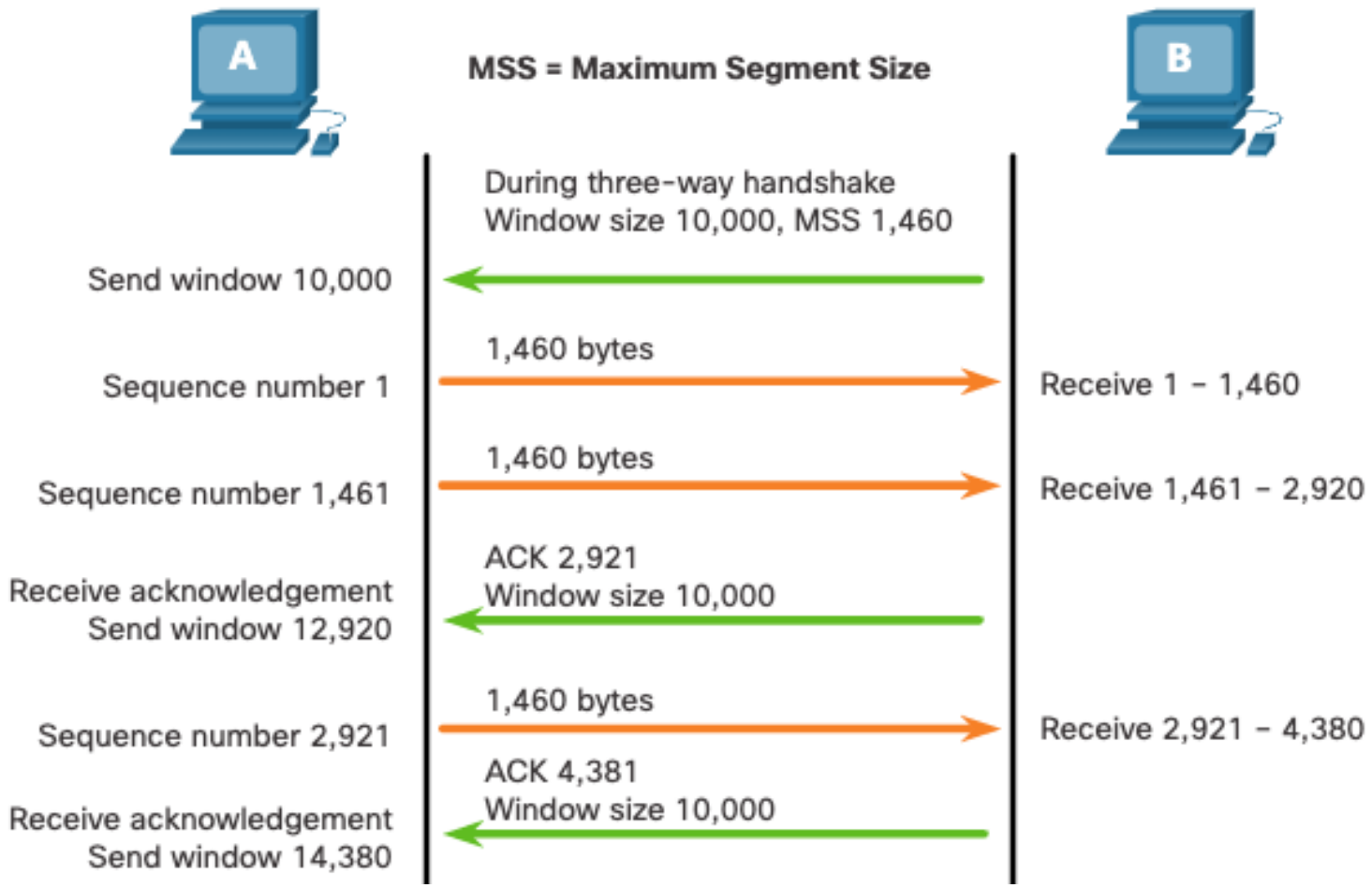
Buffering

- TCP is responsible for buffering data and determining when it is time to send a datagram.
- It is possible for an application to tell TCP to send the data it has buffered without waiting for a buffer to fill up.

TCP Flow Control – Window Size and Acknowledgments

TCP also provides mechanisms for flow control as follows:

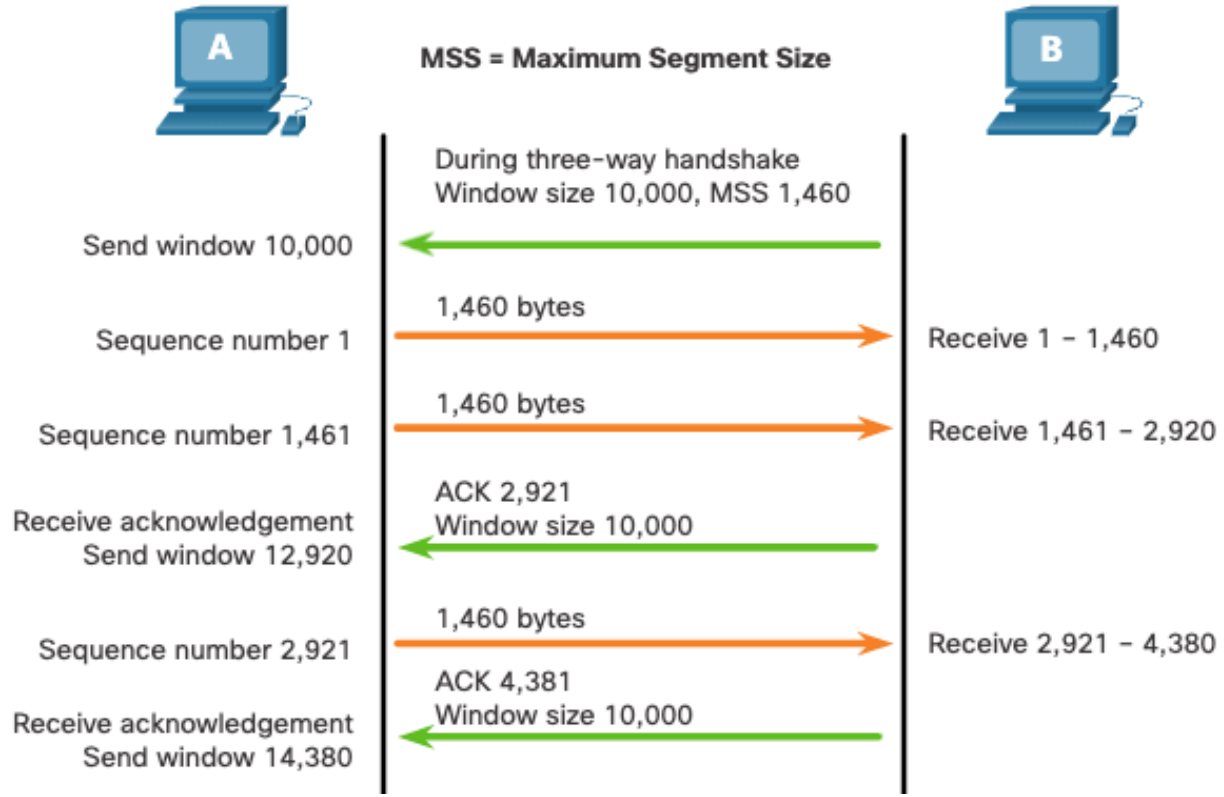
- Flow control is the amount of data that the destination can receive and process reliably.
- Flow control helps maintain the reliability of TCP transmission by adjusting the rate of data flow between source and destination for a given session.

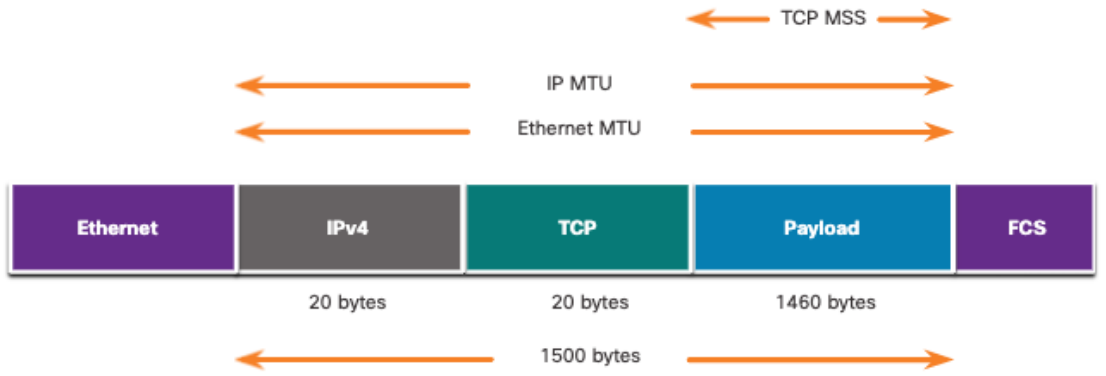


TCP Flow Control – Maximum Segment Size

Maximum Segment Size (MSS) is the maximum amount of data that the destination device can receive.

- A common MSS is 1,460 bytes when using IPv4.
- A host determines the value of its MSS field by subtracting the IP and TCP headers from the Ethernet maximum transmission unit (MTU), which is 1500 bytes be default.
- 1500 minus 40 (20 bytes for the IPv4 header and 20 bytes for the TCP header) leaves 1460 bytes.

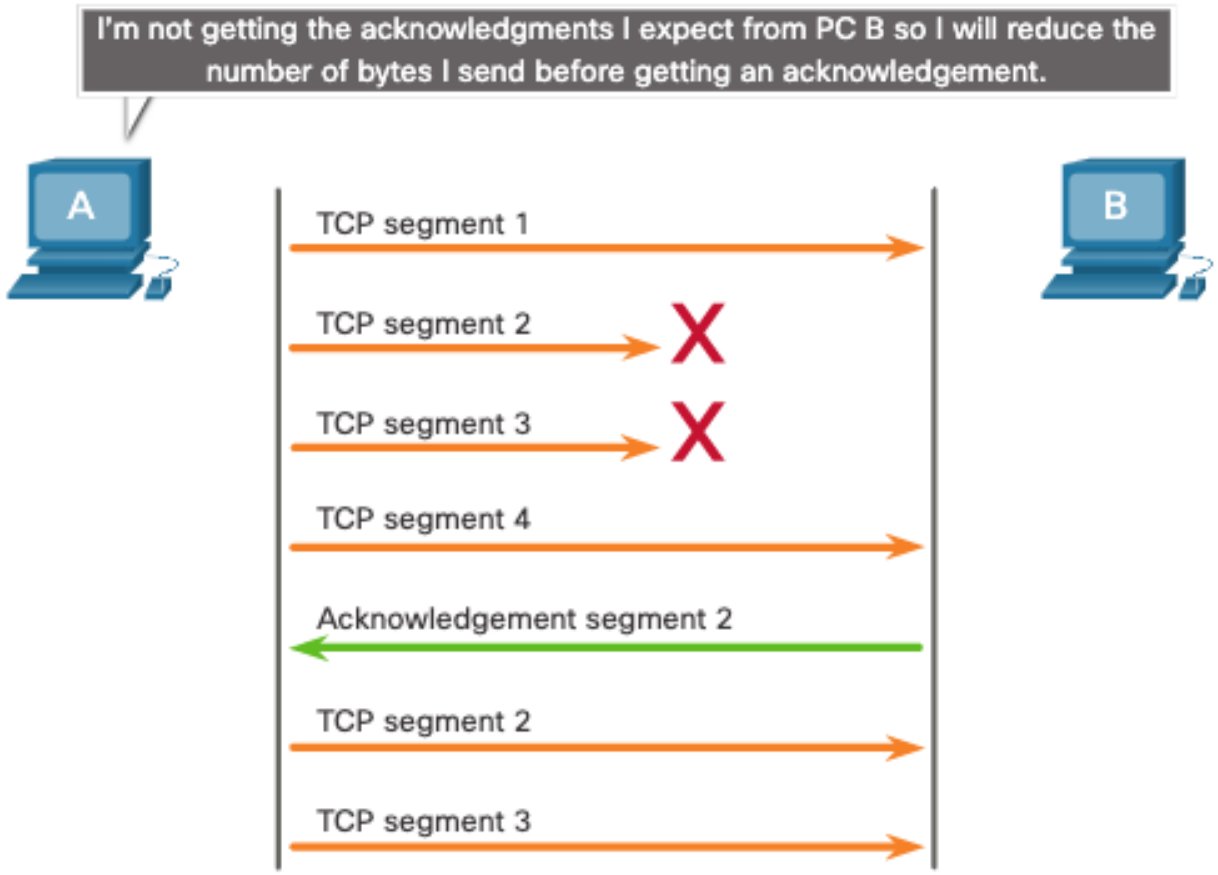




TCP Flow Control – Congestion Avoidance

When congestion occurs on a network, it results in packets being discarded by the overloaded router.

To avoid and control congestion, TCP employs several congestion handling mechanisms, timers, and algorithms.



TCP TIME_WAIT

- Once a TCP connection has been terminated (the last ACK sent) there is some unfinished business:
  - What if the ACK is lost? The last FIN will be resent, and it must be ACK'd.
  - What if there are lost or duplicated segments that finally reach the destination after a long delay?

- TCP hangs out for a while to handle these situations.

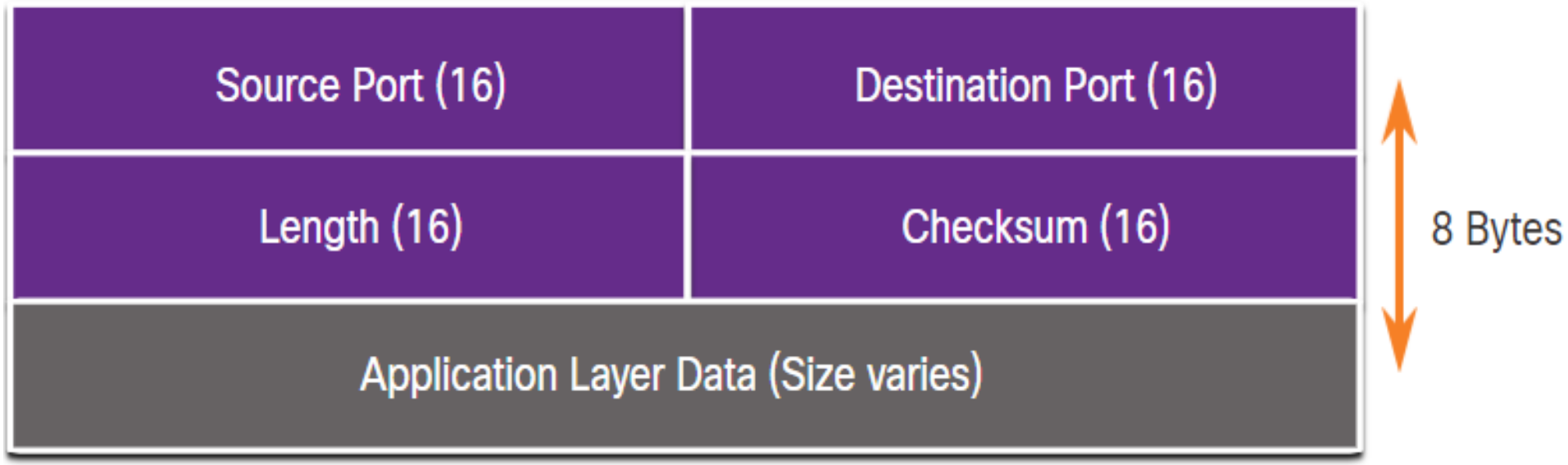
UDP Overview and Communication Process

UDP Features and Datagram Format

UDP features include the following:

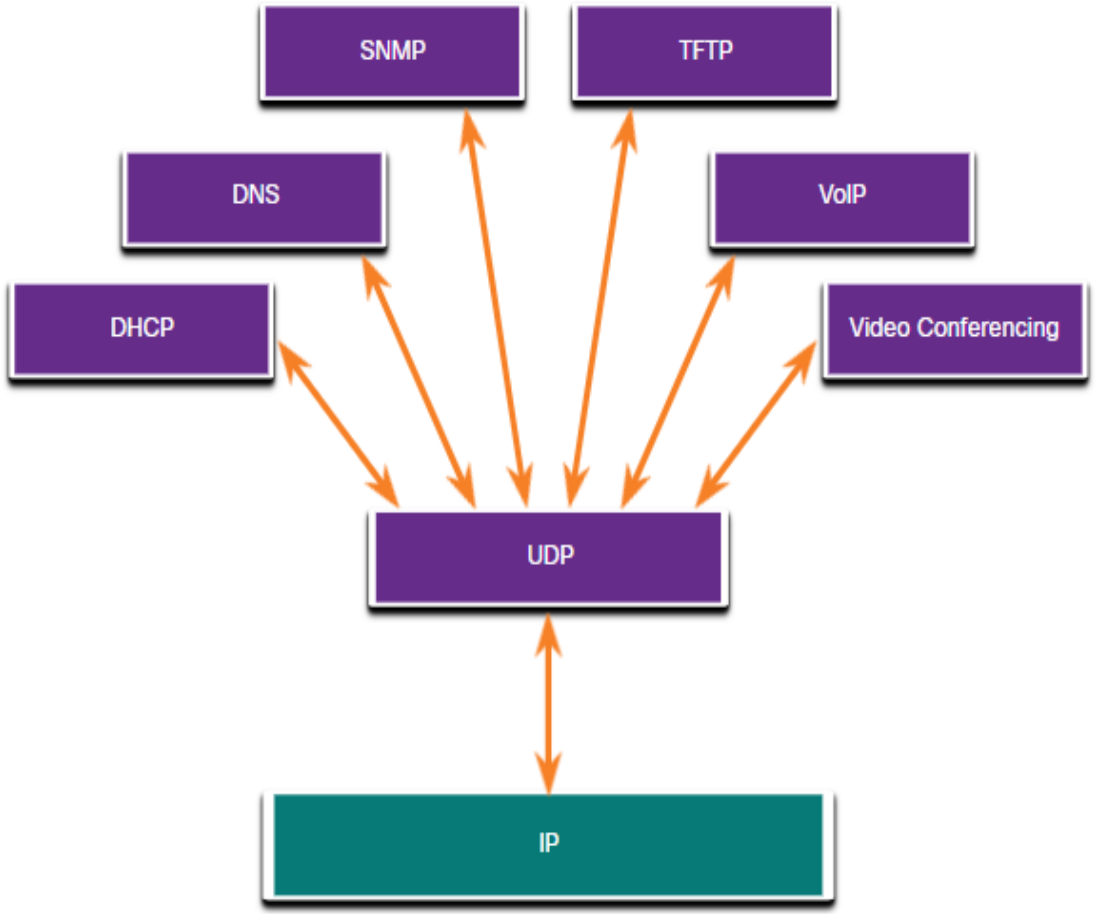
- Data is reconstructed in the order that it is received.
- Any segments that are lost are not resent.
- There is no session establishment.
- The sending is not informed about resource availability.

The UDP header is far simpler than the TCP header because it only has four fields and requires 8 bytes (i.e. 64 bits).



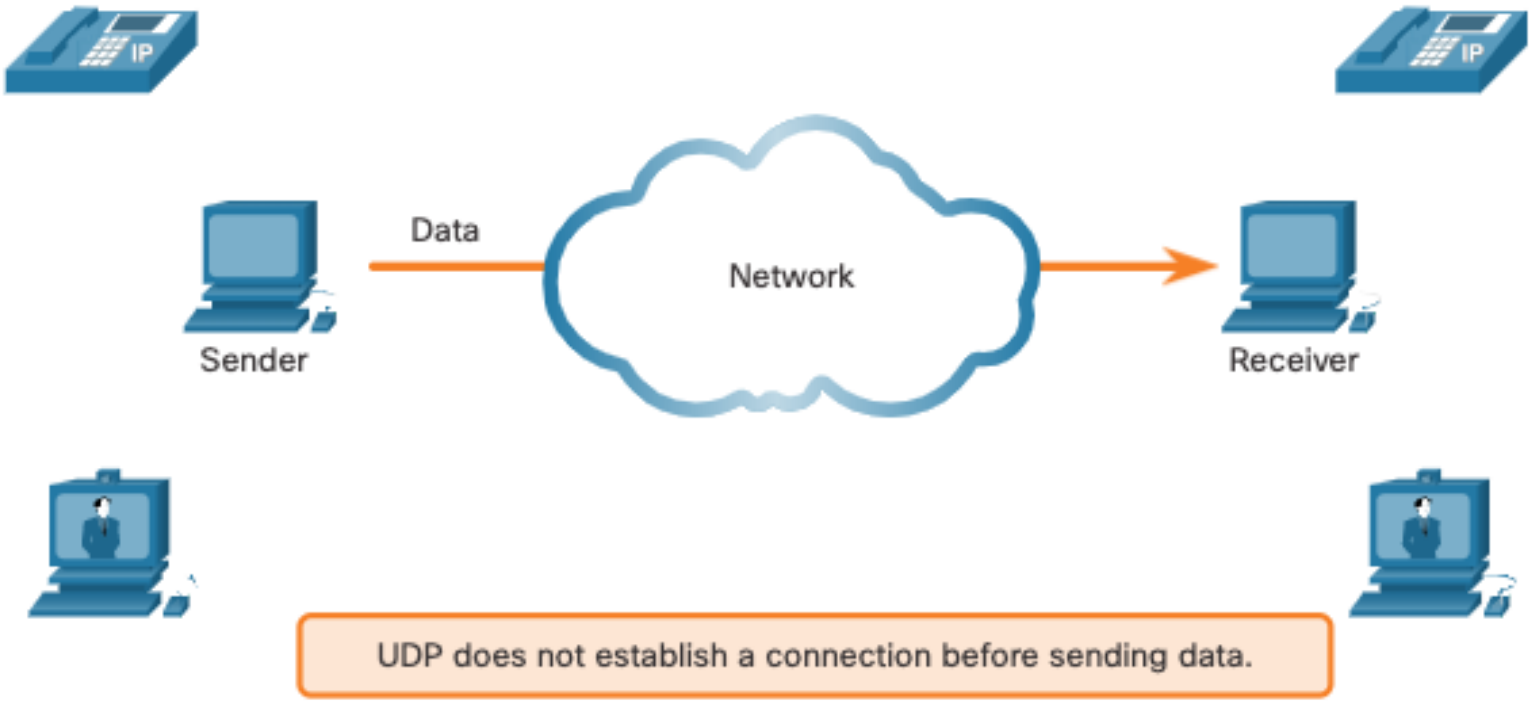
Applications that use UDP

- Live video and multimedia applications - These applications can tolerate some data loss but require little or no delay. Examples include VoIP and live streaming video.
- Simple request and reply applications - Applications with simple transactions where a host sends a request and may or may not receive a reply. Examples include DNS and DHCP.
- Applications that handle reliability themselves - Unidirectional communications where flow control, error detection, acknowledgments, and error recovery is not required, or can be handled by the application. Examples include SNMP and TFTP.



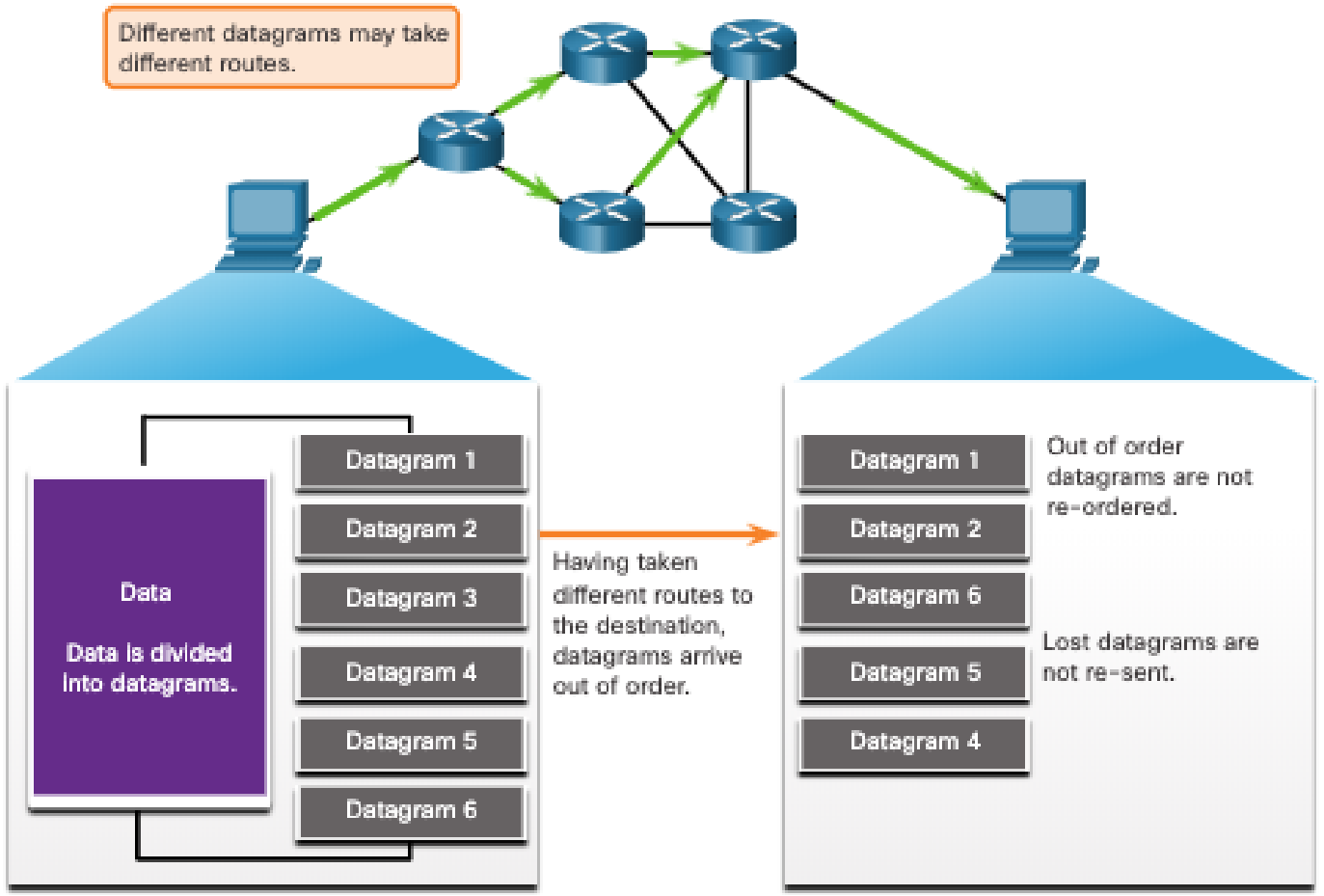
UDP Lower overhead versus Reliability

UDP does not establish a connection. UDP provides low overhead data transport because it has a small datagram header and no network management traffic.



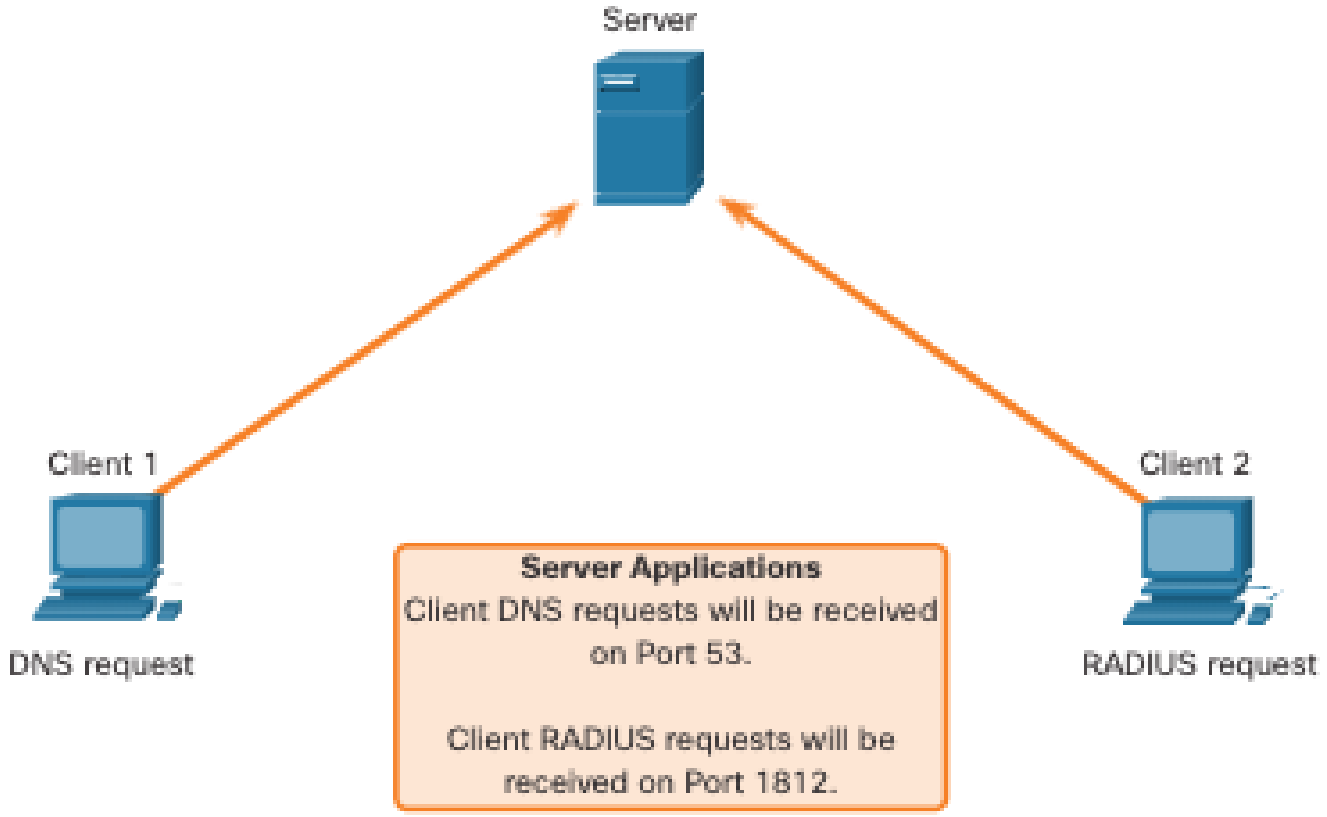
UDP Datagram Reassembly

- UDP does not track sequence numbers the way TCP does.
- UDP has no way to reorder the datagrams into their transmission order.
- UDP simply reassembles the data in the order that it was received and forwards it to the application.



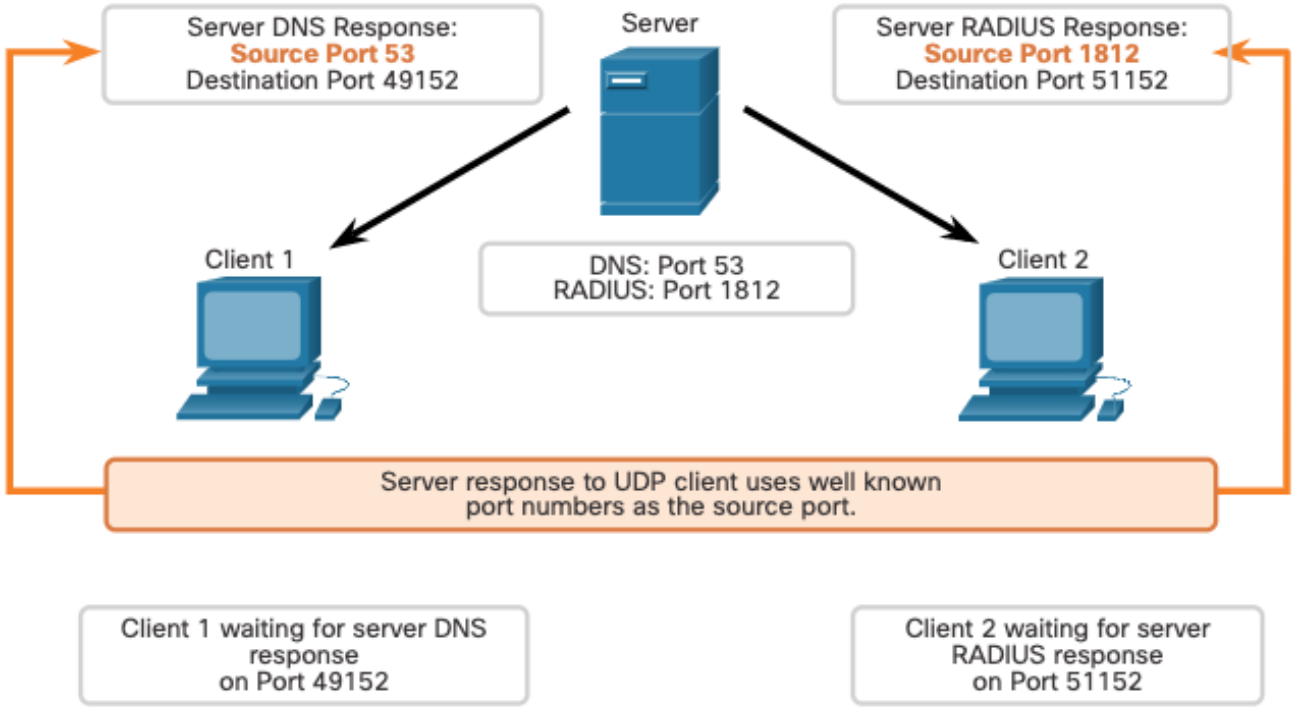
UDP Server Processes and Requests

- UDP-based server applications are assigned well-known or registered port numbers.
- UDP receives a datagram destined for one of these ports, it forwards the application data to the appropriate application based on its port number.



UDP Client Processes

- The UDP client process dynamically selects a port number from the range of port numbers and uses this as the source port for the conversation.
- The destination port is usually the well-known or registered port number assigned to the server process.
- After a client has selected the source and destination ports, the same pair of ports are used in the header of all datagrams in the transaction.



Q1

- Why is a 3-way handshake necessary?
  - HINTS: TCP is a reliable service, IP delivers each TCP segment, IP is not reliable.
- Who sends the first FIN - the server or the client?
- Once the connection is established, what is the difference between the operation of the server's TCP layer and the client's TCP layer?